

Financial Risk Analysis with Python - Blackrock

(Approach, Report Summary and Insights)

Using Jupyter Notebook as the IDE in this project

So, first we upload the dataset in the Jupyter Notebook

Now, importing Numpy and Pandas library for data cleaning and pre-processing

Loading the Dataset (CSV file) in the notebook:

```
df = pd.read_csv("blackrock.csv")
```

Viewing the Dataset:

```
df.head()
```

Checking Dataset Shape:

```
df.shape  
(800, 15)
```

Checking Column Names:

```
df.columns  
Index(['TransactionID', 'CustomerID', 'AccountID', 'AccountType',  
       'TransactionType', 'Product', 'Firm', 'Region', 'Manager',  
       'TransactionDate', 'TransactionAmount', 'AccountBalance', 'RiskScore',  
       'CreditRating', 'TenureMonths'],  
      dtype='object')
```

Checking Data Types:

```
df.info()  


| #  | Column            | Non-Null Count | Dtype   |
|----|-------------------|----------------|---------|
| 0  | TransactionID     | 800 non-null   | int64   |
| 1  | CustomerID        | 800 non-null   | object  |
| 2  | AccountID         | 800 non-null   | object  |
| 3  | AccountType       | 800 non-null   | object  |
| 4  | TransactionType   | 800 non-null   | object  |
| 5  | Product           | 800 non-null   | object  |
| 6  | Firm              | 800 non-null   | object  |
| 7  | Region            | 800 non-null   | object  |
| 8  | Manager           | 800 non-null   | object  |
| 9  | TransactionDate   | 800 non-null   | object  |
| 10 | TransactionAmount | 800 non-null   | float64 |
| 11 | AccountBalance    | 800 non-null   | float64 |
| 12 | RiskScore         | 800 non-null   | float64 |
| 13 | CreditRating      | 800 non-null   | int64   |
| 14 | TenureMonths      | 800 non-null   | int64   |

  
dtypes: float64(3), int64(3), object(9)
```

Task 1: Data Cleaning and Formatting

1.1 Remove/treat any special characters or non-numeric entries from financial fields.

Detecting anomalies in the financial fields:

```
df[['TransactionAmount', 'AccountBalance', 'RiskScore', 'CreditRating']].dtypes
```

TransactionAmount	float64
AccountBalance	float64
RiskScore	float64
CreditRating	int64
dtype:	object

No special characters or non-numeric entries were found in the financial fields.

. describe() function worked correctly

Means values are already **numeric**

```
df[['TransactionAmount', 'AccountBalance', 'RiskScore', 'CreditRating']].describe()
```

	TransactionAmount	AccountBalance	RiskScore	CreditRating
count	800.000000	800.000000	800.000000	800.000000
mean	54501.866711	72947.111696	0.466517	580.015000
std	29676.379125	34206.072059	0.233783	154.075237
min	-27916.911650	-35891.001750	-0.290309	306.000000
25%	34600.682150	51133.057895	0.312410	451.750000
50%	55524.078060	73006.751950	0.459326	575.000000
75%	73485.547887	94553.225110	0.630247	712.250000
max	165004.536900	171767.657800	1.200214	849.000000

If special characters like \$, ₹, **commas** would have appeared, we can clean them like this:

```
df['TransactionAmount'] = df['TransactionAmount'].replace('[^0-9.]', '', regex=True)
df['AccountBalance'] = df['AccountBalance'].replace('[^0-9.]', '', regex=True)
```

1.2 Convert currency amounts into numerical format.

```
df['TransactionAmount'] = pd.to_numeric(df['TransactionAmount'])
df['AccountBalance'] = pd.to_numeric(df['AccountBalance'])
df.info()
```

TransactionAmount	800 non-null	float64
AccountBalance	800 non-null	float64

1.3 Validate and format date columns.

Dates are in DD-MM-YYYY format, so converting them correctly.

```
df['TransactionDate'] = pd.to_datetime(df['TransactionDate'], dayfirst=True)
```

```
df['TransactionDate'].head()
```

```
0    2023-07-17
1    2023-01-13
2    2023-09-28
3    2023-02-05
4    2023-11-19
Name: TransactionDate, dtype: datetime64[ns]
```

1.4 Ensure account types and transaction categories are standardized.

Sometimes datasets contain inconsistent values like:

credit

Credit

CREDIT

So we need to Standardize them.

```
df['AccountType'] = df['AccountType'].str.strip().str.title()
```

```
df['TransactionType'] = df['TransactionType'].str.strip().str.title()
```

```
df[['AccountType', 'TransactionType']].head()
```

	AccountType	TransactionType
0	Credit	Withdrawal
1	Current	Payment
2	Current	Withdrawal
3	Loan	Deposit
4	Loan	Deposit

Now the data cleaning and formatting process is complete.

We can proceed with accurate financial analysis and risk modelling.

Task 2: Descriptive Transactional Analysis

2.1 Calculate monthly and yearly summaries of total credits, debits, and net transaction volume.

Key Assumption

Since dataset does not explicitly label credit/debit, we define:

- Credits → Deposit transactions (money coming in)
- Debits → Withdrawal + Payment + Transfer (money going out)

Extracting Year and Month from the transaction date

```
df['Year'] = df['TransactionDate'].dt.year
df['Month'] = df['TransactionDate'].dt.month
```

Now, Separating Credit and Debit Transactions

So, creating two new columns: Credit Amount, Debit Amount

```
df['CreditAmount'] = df.apply(
    lambda x: x['TransactionAmount'] if x['TransactionType'] == 'Deposit' else 0, axis=1)

df['DebitAmount'] = df.apply(
    lambda x: x['TransactionAmount'] if x['TransactionType'] != 'Deposit' else 0, axis=1)
```

Monthly Aggregation

```
monthly_summary = df.groupby(['Year', 'Month']).agg(
    Total_Credits = ('CreditAmount', 'sum'),
    Total_Debits = ('DebitAmount', 'sum')
).reset_index()

monthly_summary['Net_Transaction'] = (
    monthly_summary['Total_Credits'] - monthly_summary['Total_Debits']
)
```

Yearly Aggregation

```
yearly_summary = df.groupby('Year').agg(
    Total_Credits = ('CreditAmount', 'sum'),
    Total_Debits = ('DebitAmount', 'sum')
).reset_index()

yearly_summary['Net_Transaction'] = (
    yearly_summary['Total_Credits'] - yearly_summary['Total_Debits']
)
```

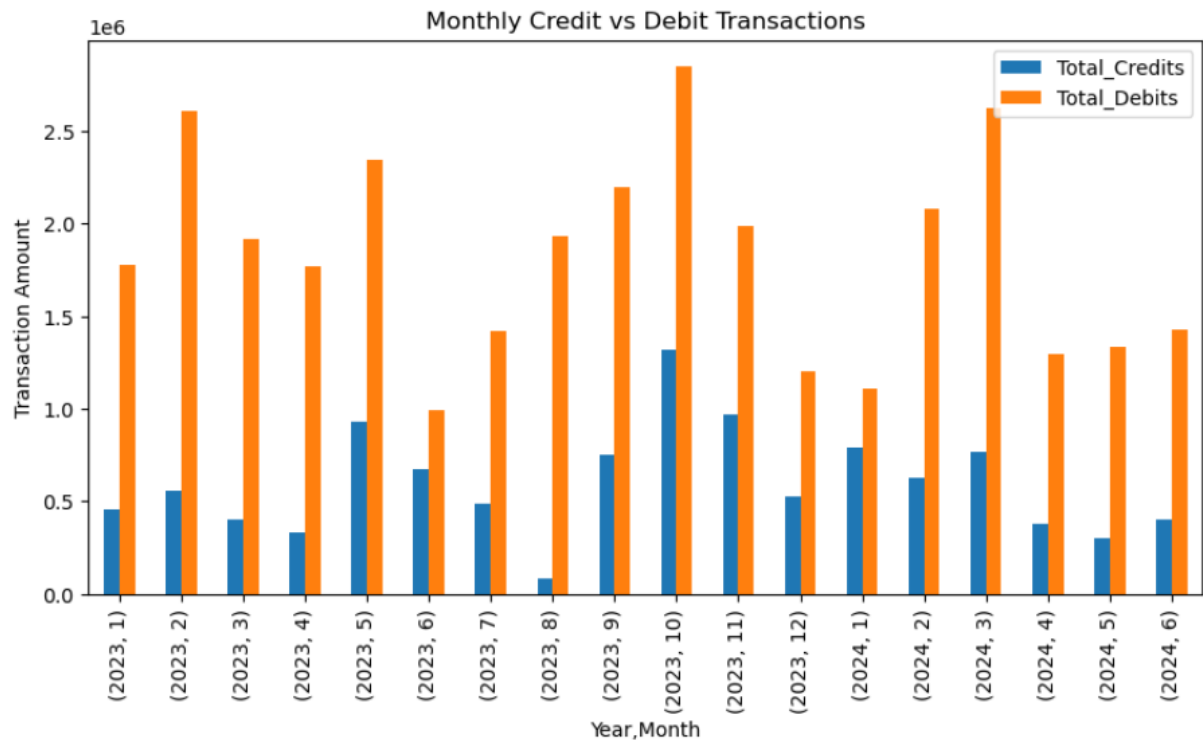
Net Transaction = Total Credits – Total Debits

Monthly Transaction Trend

```
import matplotlib.pyplot as plt

monthly_summary.set_index(['Year','Month'])[['Total_Credits','Total_Debits']].plot(kind='bar', figsize=(10,5))

plt.title('Monthly Credit vs Debit Transactions')
plt.ylabel('Transaction Amount')
plt.show()
```



Key Insights From Graph

1. Debits > Credits in all months
 - Indicates higher spending/outflow
2. Large gap between debit and credit
 - Suggests potential financial imbalance
3. Peak around Oct 2023
 - Highest activity month
4. Fluctuating pattern
 - No stable trend → irregular behavior

2.2 Plot trends in total credits vs. debits over time.

Using Monthly Summary

Creating a Proper Time Axis

We combine Year + Month into a single column for plotting.

```
monthly_summary['Date'] = pd.to_datetime(
    monthly_summary[['Year', 'Month']].assign(DAY=1)
)
```

Plotting Credit vs Debit Trends

```
import matplotlib.pyplot as plt

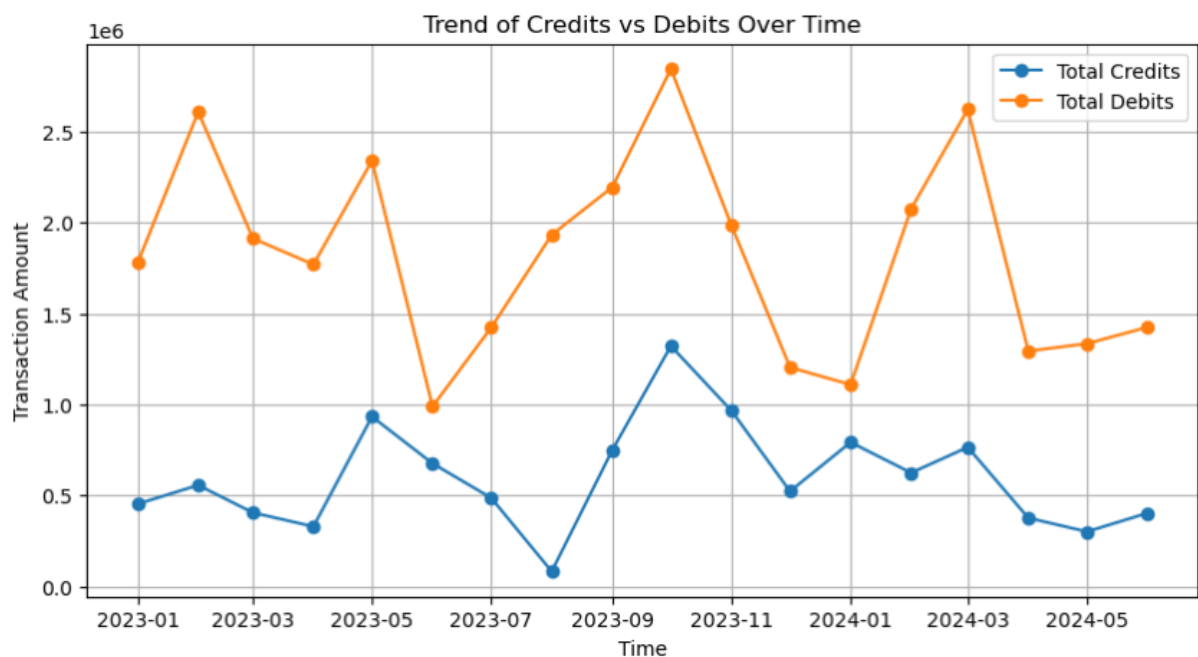
plt.figure(figsize=(10,5))

plt.plot(monthly_summary['Date'], monthly_summary['Total_Credits'], marker='o', label='Total Credits')
plt.plot(monthly_summary['Date'], monthly_summary['Total_Debits'], marker='o', label='Total Debits')

plt.title('Trend of Credits vs Debits Over Time')
plt.xlabel('Time')
plt.ylabel('Transaction Amount')

plt.legend()
plt.grid()

plt.show()
```



Key Observations

1. Debits consistently exceed credits

Debit transactions remain higher than credit transactions across most months, indicating continuous financial outflow.

2. Large gap between inflow and outflow

A significant difference between credits and debits suggests strong cash outflow and potential financial imbalance.

3. Peak spending periods observed

Spikes in debit transactions during early 2023, late 2023, and early 2024 indicate periods of high financial activity or spending.

4. Fluctuating credit patterns

Credit transactions show variability over time, reflecting irregular inflows and inconsistent income behavior.

5. Low credit periods indicate risk

Sharp drops in credit transactions (especially mid-2023) suggest reduced inflow and possible increased dependence on withdrawals.

2.3 Identify top and bottom performing accounts based on net inflow.

Net Inflow = Total Credits – Total Debits

Aggregating Data at Account Level

We will group by AccountID.

Because we want performance per account (not per transaction)

```
account_summary = df.groupby('AccountID').agg(  
    Total_Credits = ('CreditAmount', 'sum'),  
    Total_Debits = ('DebitAmount', 'sum')  
) .reset_index()
```

Calculating Net Inflow

```
account_summary['Net_Inflow'] = (  
    account_summary['Total_Credits'] - account_summary['Total_Debits']  
)
```

Identifying Top Performing Accounts

```
top_accounts = account_summary.sort_values(by='Net_Inflow', ascending=False).head(10)  
top_accounts
```

	AccountID	Total_Credits	Total_Debits	Net_Inflow
170	ACC88286	284230.60079	0.000000	284230.600790
24	ACC21719	281795.51780	7066.332080	274729.185720
102	ACC51200	251699.76008	50230.027640	201469.732440
43	ACC28295	262087.21231	62636.519390	199450.692920
163	ACC83269	180850.72689	0.000000	180850.726890
169	ACC88252	183519.24882	3893.024787	179626.224033
171	ACC88449	184286.91704	22156.027280	162130.889760
58	ACC32627	141764.34671	45164.130600	96600.216110
18	ACC18177	156307.38989	61506.258200	94801.131690
189	ACC99117	92159.60075	0.000000	92159.600750

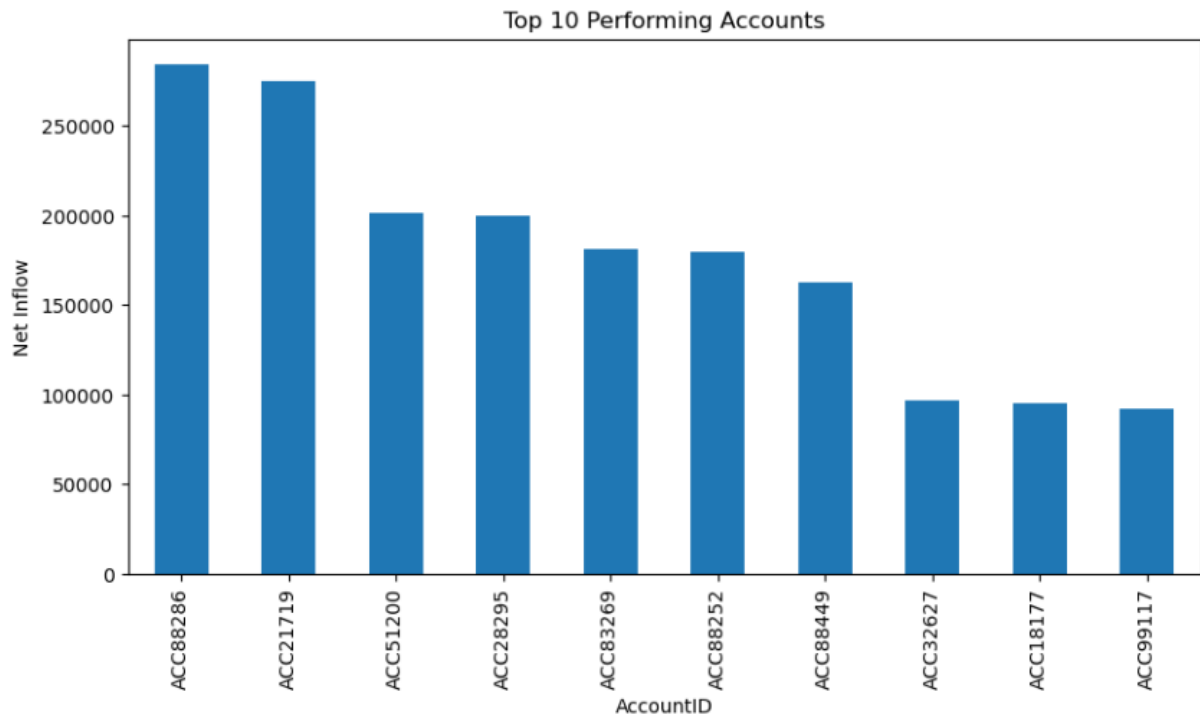
Identifying Bottom Performing Accounts

```
bottom_accounts = account_summary.sort_values(by='Net_Inflow', ascending=True).head(10)
bottom_accounts
```

	AccountID	Total_Credits	Total_Debits	Net_Inflow
83	ACC45101	35224.62786	475972.231650	-440747.603790
134	ACC70460	0.00000	432195.230130	-432195.230130
188	ACC97411	0.00000	416659.899393	-416659.899393
106	ACC52650	0.00000	387133.819950	-387133.819950
148	ACC77592	0.00000	383553.317399	-383553.317399
66	ACC35419	32955.23345	398856.436650	-365901.203200
153	ACC78581	0.00000	357789.158391	-357789.158391
184	ACC95774	0.00000	357669.637940	-357669.637940
138	ACC71938	0.00000	353666.209440	-353666.209440
157	ACC81631	0.00000	352890.598660	-352890.598660

Visualizing Top 10 Performing Accounts

```
top_accounts.set_index('AccountID')['Net_Inflow'].plot(kind='bar', figsize=(10,5))  
plt.title('Top 10 Performing Accounts')  
plt.ylabel('Net Inflow')  
plt.show()
```



Key Observations (Top & Bottom Accounts)

1. Top-performing accounts show strong positive inflow

The top accounts have significantly higher credits than debits, resulting in strong positive net inflow.

2. Some top accounts have zero debits

Several top-performing accounts have no debit transactions, indicating pure inflow and strong financial stability.

3. Bottom-performing accounts show heavy negative inflow

Bottom accounts have very high debits compared to credits, leading to large negative net inflow.

4. Multiple accounts have zero credits but high debits

Many low-performing accounts show no deposits but continuous withdrawals, indicating financial stress or risk.

5. Clear gap between top and bottom performers

There is a substantial difference between highest and lowest net inflow values, highlighting unequal financial behavior across accounts.

2.4 Identify and flag accounts as dormant or inactive if there is a gap of two months or more between consecutive transactions.

In finance, inactivity = potential risk or churn

Rule: If gap ≥ 60 days $\rightarrow$ Account is Dormant

Sorting Data Properly

We must sort transactions by: AccountID, TransactionDate

Because we need to compare previous vs next transaction

```
df = df.sort_values(by=['AccountID', 'TransactionDate'])
```

Calculating Previous Transaction Date

```
df['Prev_TransactionDate'] = df.groupby('AccountID')['TransactionDate'].shift(1)
```

Calculating Gap (in Days)

```
df['Gap_Days'] = (df['TransactionDate'] - df['Prev_TransactionDate']).dt.days
```

Flaging Dormant Transactions

1 $\rightarrow$ Dormant

0 $\rightarrow$ Active

```
df['Dormant_Flag'] = df['Gap_Days'].apply(lambda x: 1 if x >= 60 else 0)
```

Identifying Dormant Accounts

```
dormant_accounts = df[df['Dormant_Flag'] == 1]['AccountID'].unique()
dormant_accounts
```

Creating Account-Level Status

```
account_status = df.groupby('AccountID')['Dormant_Flag'].max().reset_index()

account_status['Account_Status'] = account_status['Dormant_Flag'].apply(
    lambda x: 'Dormant' if x == 1 else 'Active'
)

account_status
```

	AccountID	Dormant_Flag	Account_Status
0	ACC10117	1	Dormant
1	ACC10996	1	Dormant
2	ACC11062	1	Dormant
3	ACC11188	1	Dormant
4	ACC11285	1	Dormant
...	...	...	...
187	ACC97225	1	Dormant
188	ACC97411	1	Dormant
189	ACC99117	0	Active
190	ACC99409	1	Dormant
191	ACC99549	1	Dormant

192 rows × 3 columns

Count Dormant vs Active

```
account_status['Account_Status'].value_counts()
```

```
Account_Status
Dormant      172
Active        20
```

Percentage

```
account_status['Account_Status'].value_counts(normalize=True) * 100
```

```
Account_Status
Dormant      89.583333
Active       10.416667
```

Pie Chart

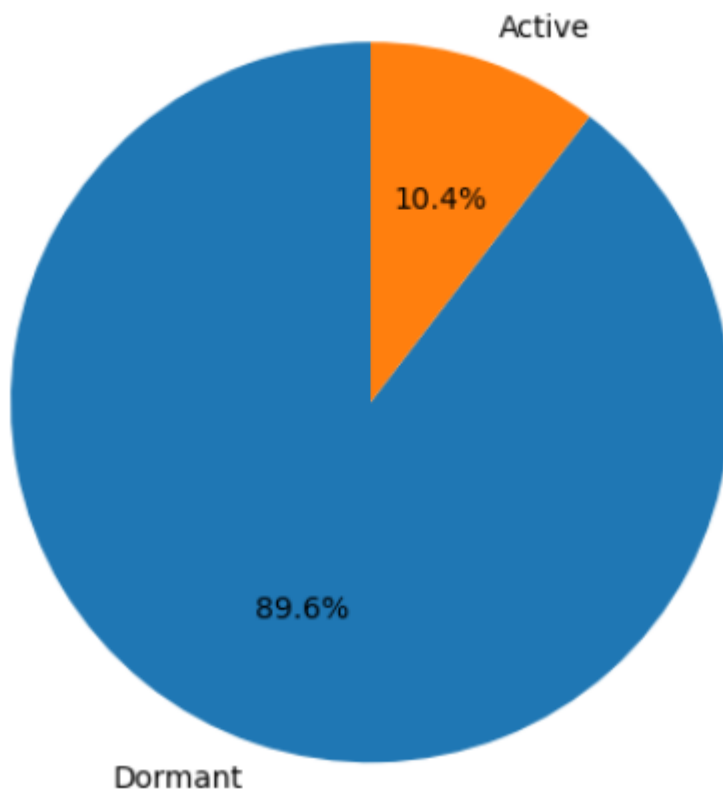
```
counts = account_status['Account_Status'].value_counts()

# Pie chart
plt.figure()
plt.pie(counts,
        labels=counts.index,
        autopct='%1.1f%%',
        startangle=90)

plt.title('Account Status Distribution (Dormant vs Active)')
plt.axis('equal') # Makes circle perfect

plt.show()
```

Account Status Distribution (Dormant vs Active)



Critical Observation

Almost 90% accounts are dormant

This is **very high**

This suggests:

- Transactions are **not frequent**
- Many accounts have **large gaps (>60 days)**
- Dataset shows **low engagement behavior**

But this is not realistically happen in real banking

In real banking:

Dormant accounts $\approx$ 10–30% NOT 90%

So, our logic is correct

But threshold may be **too strict**

Interpretation

A total of 172 accounts (89.58%) were classified as dormant, while only 20 accounts (10.42%) were active. This indicates that a large proportion of accounts have gaps of more than 60 days between transactions, suggesting low transaction frequency and reduced customer engagement within the dataset.

Critical Insight

The high percentage of dormant accounts may be due to the strict threshold of 60 days used for classification or the nature of the dataset, where transactions are not frequent.

Conclusion

The dormancy analysis reveals that the majority of accounts are inactive based on the defined threshold. While this highlights potential inactivity and engagement issues, the results also suggest that the dormancy criteria may be stringent for this dataset. Adjusting the threshold or incorporating additional activity metrics could provide a more balanced classification.

Task 2: Descriptive Transactional Analysis (*Summary*)

Key Insights

1. Debit transactions consistently exceed credits, with peak debits reaching ~2.8M vs ~1.3M credits.
2. Net transactions remain negative across most months, indicating continuous financial outflow.
3. Transaction patterns fluctuate, with highest activity observed in late 2023 and early 2024.
4. Top accounts show strong positive inflow (up to ~284K), while bottom accounts show heavy losses (up to ~-440K).
5. A significant majority of accounts (~89.6%) are dormant, indicating low engagement.

Business Recommendations

1. Improve customer engagement to reduce high dormancy levels.
2. Monitor high debit accounts to manage financial risk and outflow behavior.
3. Target top-performing accounts with premium services and retention strategies.
4. Identify and support low-performing accounts to reduce financial stress.
5. Re-evaluate dormancy threshold for more realistic classification.

Overall Conclusion

The descriptive transactional analysis reveals a pattern of higher financial outflows, uneven customer activity, and a large proportion of inactive accounts. These findings highlight the need for improved customer engagement, better risk monitoring, and targeted strategies for different customer segments to enhance financial performance and reduce potential risk.

Task 3: Customer Profile Building

3.1 Group accounts by activity levels: High, Medium, Low based on transaction frequency on your analysis and rubrics. Do not forget to mention the rubric in the headings.

We measure: Transaction Frequency = Number of transactions per account

Calculating Transaction Frequency per account

```
account_activity = df.groupby('AccountID').size().reset_index(name='Transaction_Count')
```

Defining Rubric

Activity Level	Condition
High	$\geq$ 75th percentile
Medium	25th–75th percentile
Low	$\leq$ 25th percentile

Calculating Percentiles

```
q25 = account_activity['Transaction_Count'].quantile(0.25)
q75 = account_activity['Transaction_Count'].quantile(0.75)
```

Assigning Activity Levels

```
def classify_activity(x):
    if x >= q75:
        return 'High'
    elif x <= q25:
        return 'Low'
    else:
        return 'Medium'

account_activity['Activity_Level'] = account_activity['Transaction_Count'].apply(classify_activity)
```

```
account_activity.head()
```

	AccountID	Transaction_Count	Activity_Level
0	ACC10117	6	High
1	ACC10996	4	Medium
2	ACC11062	6	High
3	ACC11188	9	High
4	ACC11285	5	High

```
account_activity['Activity_Level'].value_counts()
```

```
Activity_Level
Low      79
High     65
Medium   48
Name: count, dtype: int64
```

The result shows:

Low > High > Medium

This is slightly **imbalanced**, because:

In percentile-based segmentation, we expect:

- ~25% Low
- ~50% Medium
- ~25% High

But our result:

- Low is highest
- Medium is lowest

This likely happened because:

Our data has **many repeated transaction counts**

Example: Many accounts might have:

- 5 transactions
- 6 transactions

So, percentile split becomes uneven.

We can improve this segmentation

By using Ranking (Better split)

```
account_activity['Rank'] = account_activity['Transaction_Count'].rank(pct=True)

def classify(x):
    if x >= 0.75:
        return 'High'
    elif x <= 0.25:
        return 'Low'
    else:
        return 'Medium'

account_activity['Activity_Level'] = account_activity['Rank'].apply(classify)
```

```
account_activity['Activity_Level'].value_counts()
```

```
Activity_Level
Medium    105
High       45
Low        42
```

```
account_activity.head()
```

	AccountID	Transaction_Count	Activity_Level	Rank
0	ACC10117	6	High	0.812500
1	ACC10996	4	Medium	0.539062
2	ACC11062	6	High	0.812500
3	ACC11188	9	High	0.984375
4	ACC11285	5	Medium	0.716146

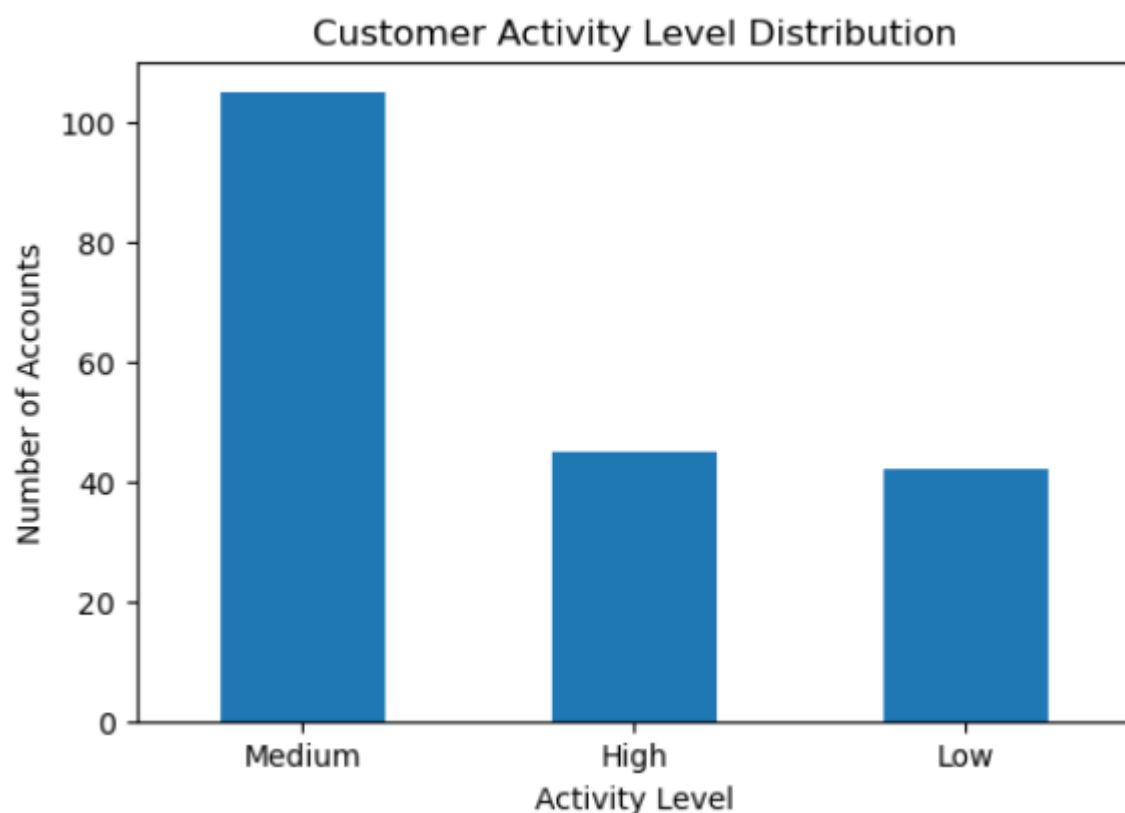
I initially used percentile-based segmentation, but I observed skewness due to repeated transaction counts. To improve segmentation quality, I explored rank-based approach.

Now, visualizing these segment

```
import matplotlib.pyplot as plt

account_activity['Activity_Level'].value_counts().plot(
    kind='bar',
    figsize=(6,4)
)

plt.title('Customer Activity Level Distribution')
plt.xlabel('Activity Level')
plt.ylabel('Number of Accounts')
plt.xticks(rotation=0)
plt.show()
```



Key Observation (Activity Level Distribution)

1. Majority accounts are Medium activity (105 accounts, ~54.7%)

This shows most customers have moderate transaction frequency.

2. High activity accounts are 45 (~23.4%)

A smaller segment of customers are highly active and frequently transact.

3. Low activity accounts are 42 (~21.9%)

A similar proportion of customers show low engagement with minimal transactions.

Task 3.2 Segment customers by average balance and transaction volume.

To segment customers based on:

- **Average Account Balance** → financial strength
- **Transaction Volume** → activity level

Average Balance = Mean of AccountBalance per account

Transaction Volume = Total Transaction Amount per account

Aggregating at Account Level

```
account_profile = df.groupby('AccountID').agg(  
    Avg_Balance = ('AccountBalance', 'mean'),  
    Total_Transaction = ('TransactionAmount', 'sum')  
) .reset_index()
```

Defining Rubric

We will use percentile-based segmentation.

For both metrics:

Level	Condition
High	≥ 75th percentile
Medium	25th–75th percentile
Low	≤ 25th percentile

Calculating Percentiles

```
bal_q25 = account_profile['Avg_Balance'].quantile(0.25)  
bal_q75 = account_profile['Avg_Balance'].quantile(0.75)  
  
txn_q25 = account_profile['Total_Transaction'].quantile(0.25)  
txn_q75 = account_profile['Total_Transaction'].quantile(0.75)
```

Classify Balance

```
def balance_segment(x):  
    if x >= bal_q75:  
        return 'High Balance'  
    elif x <= bal_q25:  
        return 'Low Balance'  
    else:  
        return 'Medium Balance'  
  
account_profile['Balance_Segment'] = account_profile['Avg_Balance'].apply(balance_segment)
```

Classify Transaction Volume

```
def txn_segment(x):
    if x >= txn_q75:
        return 'High Volume'
    elif x <= txn_q25:
        return 'Low Volume'
    else:
        return 'Medium Volume'

account_profile['Transaction_Segment'] = account_profile['Total_Transaction'].apply(txn_segment)
```

Combining Segments

```
account_profile['Customer_Segment'] = (
    account_profile['Balance_Segment'] + ' | ' + account_profile['Transaction_Segment']
)
```

```
account_profile.head()
```

	AccountID	Avg_Balance	Total_Transaction	Balance_Segment	Transaction_Segment	Customer_Segment
0	ACC10117	81106.206188	246887.286800	Medium Balance	Medium Volume	Medium Balance Medium Volume
1	ACC10996	101733.898198	358817.022010	High Balance	High Volume	High Balance High Volume
2	ACC11062	57108.763888	328776.968270	Low Balance	High Volume	Low Balance High Volume
3	ACC11188	80235.151167	386419.024114	Medium Balance	High Volume	Medium Balance High Volume
4	ACC11285	71205.645238	284440.530857	Medium Balance	Medium Volume	Medium Balance Medium Volume

Segments

Segment

Meaning

High Balance High Volume	Premium customers
High Balance Low Volume	Wealthy but inactive
Low Balance High Volume	Active but low funds
Low Balance Low Volume	Low engagement

```
account_profile['Customer_Segment'].value_counts()
```

Customer_Segment	
Medium Balance Medium Volume	49
Medium Balance High Volume	25
Low Balance Medium Volume	24
High Balance Medium Volume	23
Medium Balance Low Volume	22
High Balance Low Volume	16
Low Balance High Volume	14
Low Balance Low Volume	10
High Balance High Volume	9

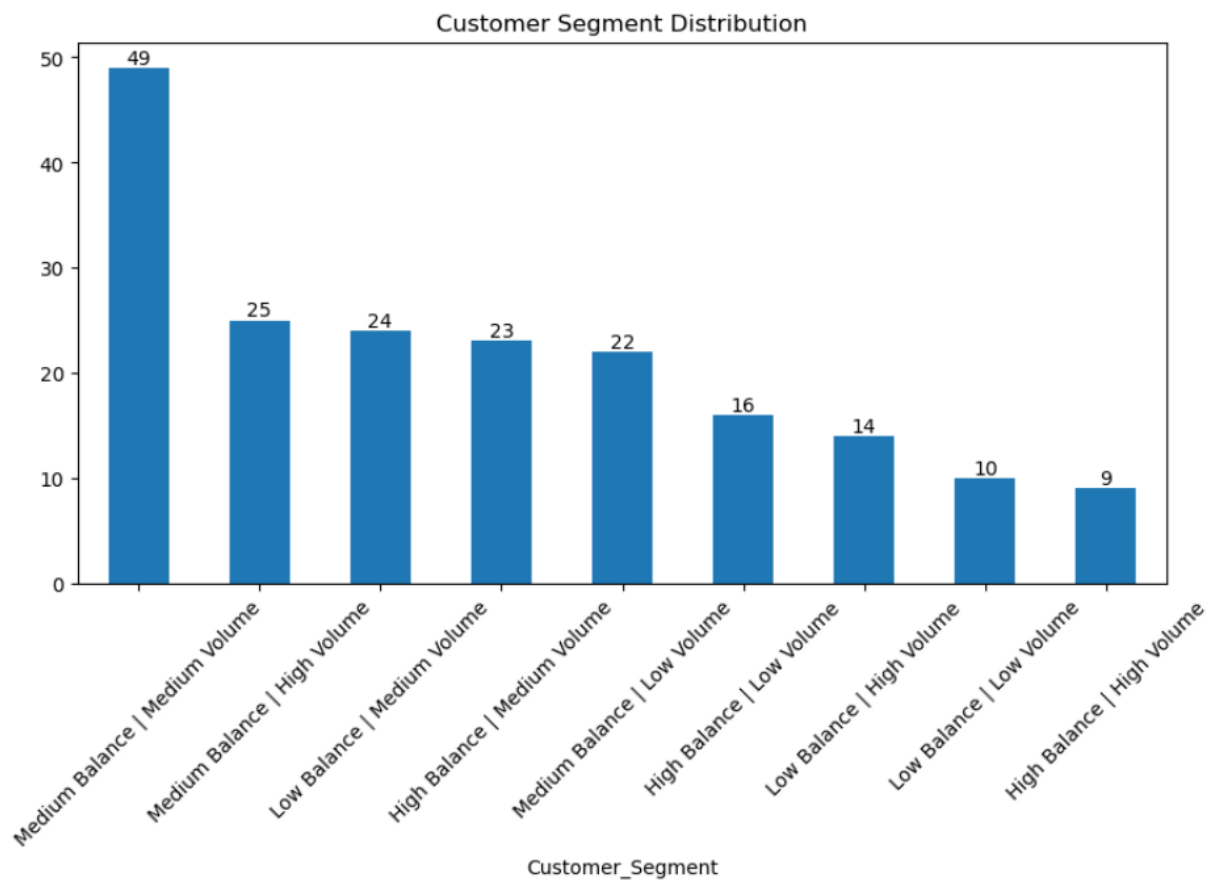
```
pd.crosstab(account_profile['Balance_Segment'],
            account_profile['Transaction_Segment'])
```

Transaction_Segment	High Volume	Low Volume	Medium Volume
Balance_Segment			
High Balance	9	16	23
Low Balance	14	10	24
Medium Balance	25	22	49

Now, visualizing this segment

```
ax = account_profile['Customer_Segment'].value_counts().plot(kind='bar', figsize=(10,5))
# Adding data labels
ax.bar_label(ax.containers[0])

plt.xticks(rotation=45)
plt.title('Customer Segment Distribution')
plt.show()
```



Key Observation

1. Largest segment: Medium Balance | Medium Volume (49 accounts, ~25.5%)

This indicates that most customers fall into a stable, moderate activity and balance category.

2. High activity mostly comes from Medium Balance customers (25 accounts)

Medium balance customers are the most active in terms of transaction volume.

3. High Balance customers are less active

Only **9 accounts** fall under *High Balance | High Volume*, while more are in medium (23) and low volume (16).

4. Low Balance customers still show high activity (14 accounts)

This suggests some customers transact frequently despite having lower balances, indicating possible financial stress or high cash flow usage.

5. Low-value segment is smallest (Low Balance | Low Volume = 10 accounts)

This group represents the least engaged and lowest financial contribution segment.

Key Insight

High balance does not necessarily correspond to high transaction activity, indicating that financially strong customers may not always be highly engaged.

3.3 Create profiles for:

High-net inflow accounts

High-frequency low-balance accounts

Accounts with negative or near-zero balances

We will build 3 profiles:

1. High-Net Inflow Accounts: Accounts with significantly higher total credits compared to debits, indicating strong financial inflow.

```
# Using account_summary from Task 2.3

high_inflow_threshold = account_summary['Net_Inflow'].quantile(0.75)

high_net_inflow_accounts = account_summary[
    account_summary['Net_Inflow'] >= high_inflow_threshold
]

high_net_inflow_accounts.head()
```

	AccountID	Total_Credits	Total_Debits	Net_Inflow
6	ACC12182	130450.262350	49923.06381	80527.198540
9	ACC15228	46141.121310	51787.06991	-5645.948600
10	ACC15359	69405.131370	61378.50949	8026.621880
18	ACC18177	156307.389890	61506.25820	94801.131690
19	ACC19156	63106.086354	94881.99586	-31775.909506

2. High-Frequency Low-Balance Accounts: Accounts with high transaction frequency but low average balance, indicating frequent usage but limited financial capacity.

```
# Merge activity + balance data

profile_df = account_profile.merge(account_activity, on='AccountID')

# Define thresholds
freq_threshold = profile_df['Transaction_Count'].quantile(0.75)
balance_threshold = profile_df['Avg_Balance'].quantile(0.25)

high_freq_low_balance = profile_df[
    (profile_df['Transaction_Count'] >= freq_threshold) &
    (profile_df['Avg_Balance'] <= balance_threshold)
]

high_freq_low_balance.head()
```

AccountID	Avg_Balance	Total_Transaction	Balance_Segment	Transaction_Segment	Customer_Segment	Transaction_Count	Activity_Level
ACC11062	57108.763888	328776.96827	Low Balance	High Volume	Low Balance High Volume	6	High
ACC24070	53811.388618	338136.69381	Low Balance	High Volume	Low Balance High Volume	5	High
ACC34431	55556.756888	464671.95707	Low Balance	High Volume	Low Balance High Volume	8	High
ACC35419	53390.330235	431811.67010	Low Balance	High Volume	Low Balance High Volume	6	High
ACC41829	61628.656517	765315.96385	Low Balance	High Volume	Low Balance High Volume	13	High

3. Negative or Near-Zero Balance Accounts: Accounts with very low or negative balances, indicating potential financial instability or overdraft risk.

```
# Define near-zero threshold (example: < 1000)

negative_balance_accounts = account_profile[
    account_profile['Avg_Balance'] <= 1000
]

negative_balance_accounts.head()
```

AccountID	Avg_Balance	Total_Transaction	Balance_Segment	Transaction_Segment	Customer_Segment
-----------	-------------	-------------------	-----------------	---------------------	------------------

Count of Each Profile

```
print("High Net Inflow:", len(high_net_inflow_accounts))
print("High Frequency Low Balance:", len(high_freq_low_balance))
print("Low / Negative Balance:", len(negative_balance_accounts))
```

```
High Net Inflow: 48
High Frequency Low Balance: 15
Low / Negative Balance: 0
```

Checking Overlapping High-Risk Accounts

```
common_accounts = set(high_freq_low_balance['AccountID']) & set(negative_balance_accounts['AccountID'])
len(common_accounts)

0
```

No “Low / Negative Balance” accounts found

Note: Since no meaningful insights were obtained using a fixed threshold of 1000, the threshold was adjusted to the 10th percentile of average balance. This percentile-based approach ensures a data-driven and more relevant identification of low or near-zero balance accounts.

```
low_balance_threshold = account_profile['Avg_Balance'].quantile(0.10)

negative_balance_accounts = account_profile[
    account_profile['Avg_Balance'] <= low_balance_threshold]
```

```
print("High Net Inflow:", len(high_net_inflow_accounts))
print("High Frequency Low Balance:", len(high_freq_low_balance))
print("Low / Negative Balance:", len(negative_balance_accounts))
```

```
High Net Inflow: 48
High Frequency Low Balance: 15
Low / Negative Balance: 20
```

High-Risk Accounts

```
common_accounts = set(high_freq_low_balance['AccountID']) & set(negative_balance_accounts['AccountID'])
len(common_accounts)
```

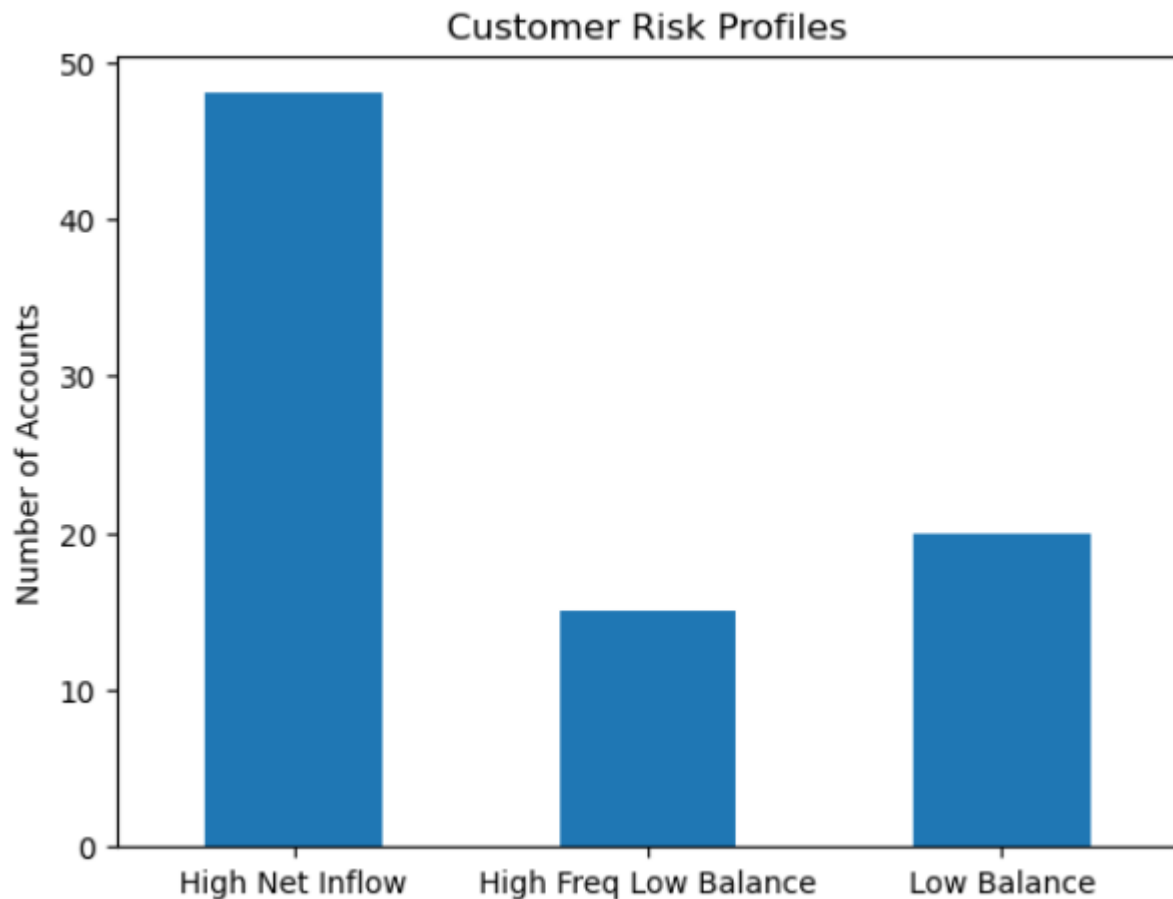
```
2
```

Now, Visualizing these profiles

```
profile_counts = {
    'High Net Inflow': 48,
    'High Freq Low Balance': 15,
    'Low Balance': 20
}

pd.Series(profile_counts).plot(kind='bar')

plt.title('Customer Risk Profiles')
plt.ylabel('Number of Accounts')
plt.xticks(rotation=0)
plt.show()
```



Key Observation

1. High Net Inflow Accounts (48, ~25%)

These accounts show strong financial inflow and represent high-value customers.

2. High Frequency Low Balance Accounts (15, ~7.8%)

These customers transact frequently despite low balances, indicating possible financial stress or heavy cash flow usage.

3. Low / Near-Zero Balance Accounts (20, ~10.4%)

These accounts have very low balances and may face liquidity issues or financial instability.

4. Very Few High-Risk Overlapping Accounts (2 accounts)

Only 2 accounts fall into both high-frequency and low-balance categories, representing the most critical risk segment.

5. Clear Separation Between Value and Risk Segments

High inflow accounts are distinct from low-balance and high-risk accounts, indicating diverse financial behaviour across customers.

Key Insight

High activity does not necessarily indicate financial strength, as some highly active accounts maintain low balances, highlighting potential financial risk.

Task 3: Customer Profile Building (Summary)

Key Insights

1. Majority accounts are Medium activity (105, ~54.7%)

Indicates most customers have moderate transaction frequency.

2. High activity accounts are limited (45, ~23.4%)

A smaller segment shows high engagement and frequent transactions.

3. Largest segment: Medium Balance | Medium Volume (49, ~25.5%)

Represents the most stable and typical customer group.

4. High Net Inflow accounts (48, ~25%) indicate strong financial customers

These accounts contribute significantly to positive cash flow.

5. Risk segments identified: 15 high-frequency low-balance & 20 low-balance accounts (~18%)

Indicates a portion of customers with potential financial stress.

Business Recommendations

1. Target medium segment (105 accounts) for engagement growth

Encourage increased activity through personalized offers.

2. Retain high-value customers (48 high inflow accounts)

Provide premium services and investment opportunities.

3. Monitor high-frequency low-balance accounts (15 accounts)

Flag for potential financial stress and risk management.

4. Support low-balance customers (20 accounts)

Offer financial advisory or credit management solutions.

5. Use segmentation for personalized marketing strategies

Different customer groups require tailored engagement approaches.

Overall Summary

The customer profiling analysis highlights a predominantly medium-activity customer base, with a balanced distribution across activity and financial segments. High-value customers contribute strong inflows, while a smaller segment exhibits potential financial risk. These insights enable targeted engagement, improved customer retention, and effective risk management strategies.

Task 4: Financial Risk Identification

4.1 Track accounts with frequent large withdrawals or overdrafts.

Large Withdrawal: A withdrawal transaction whose amount is significantly higher than normal.

Defining Threshold:

Using percentile

Large Withdrawal = Top 10% of withdrawal amounts

Frequent Large Withdrawals: Accounts that perform multiple large withdrawals.

Overdraft: When account balance becomes negative after transaction.

Filtering Withdrawal Transactions

```
withdrawals = df[df['TransactionType'] == 'Withdrawal']
```

Defining Large Withdrawal Threshold

Top 10% = large transactions

```
large_threshold = withdrawals['TransactionAmount'].quantile(0.90)
```

Identifying Large Withdrawals

```
withdrawals['Large_Withdrawal'] = withdrawals['TransactionAmount'].apply(
    lambda x: 1 if x >= large_threshold else 0
)
```

Count Large Withdrawals per Account

```
large_withdrawals_count = withdrawals.groupby('AccountID')['Large_Withdrawal'].sum().reset_index()
```

Flag Frequent Large Withdrawals

```
freq_threshold = large_withdrawals_count['Large_Withdrawal'].quantile(0.90)

frequent_large_withdrawals = large_withdrawals_count[
    large_withdrawals_count['Large_Withdrawal'] >= freq_threshold
]
```

Identifying Overdraft Accounts

```
overdraft_accounts = df[df['AccountBalance'] < 0]['AccountID'].unique()
```

Combine Risky Accounts

```
risky_accounts = set(frequent_large_withdrawals['AccountID']) | set(overdraft_accounts)
len(risky_accounts)
```

```
print("Frequent Large Withdrawal Accounts:", len(frequent_large_withdrawals))
print("Overdraft Accounts:", len(overdraft_accounts))
print("Total Risky Accounts:", len(risky_accounts))
```

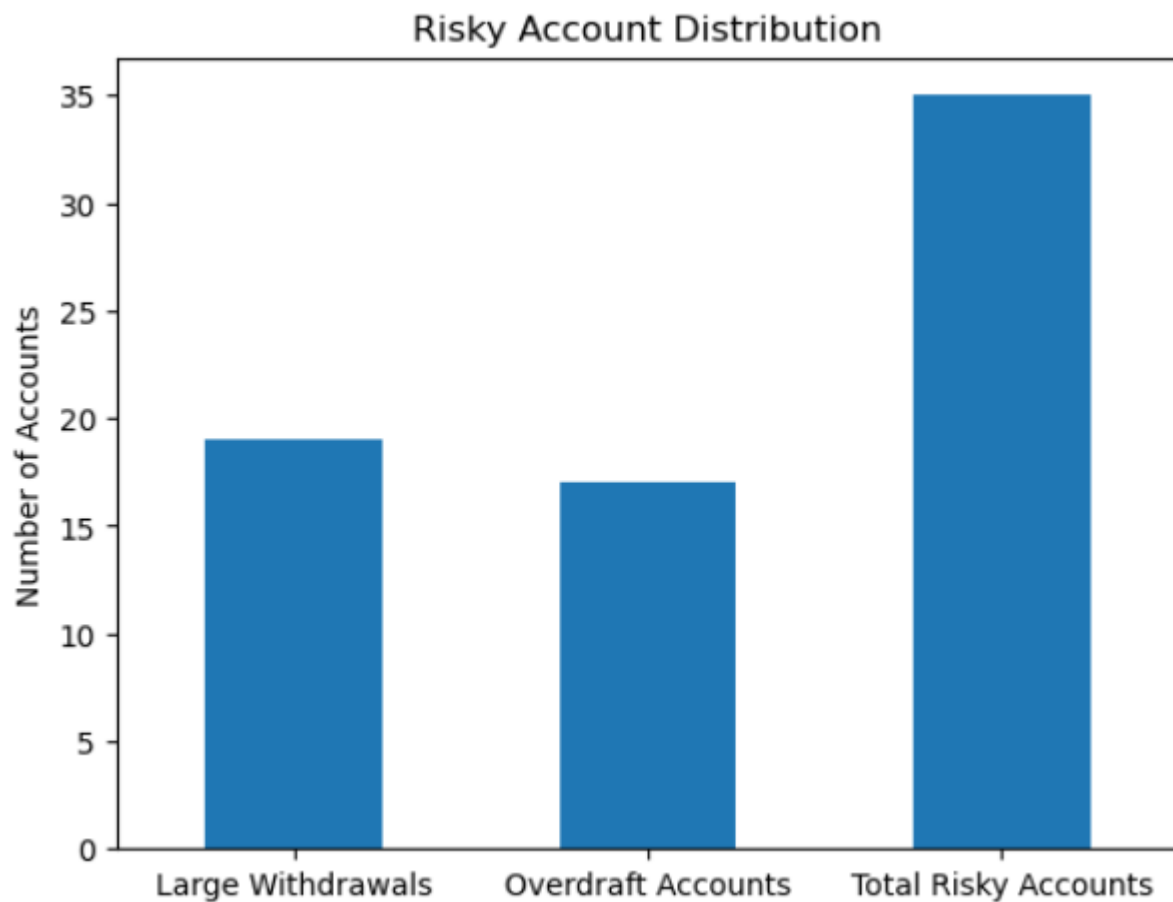
```
Frequent Large Withdrawal Accounts: 19
Overdraft Accounts: 17
Total Risky Accounts: 35
```

Visualization

```
risk_counts = {
    'Large Withdrawals': 19,
    'Overdraft Accounts': 17,
    'Total Risky Accounts': 35
}

pd.Series(risk_counts).plot(kind='bar')

plt.title('Risky Account Distribution')
plt.ylabel('Number of Accounts')
plt.xticks(rotation=0)
plt.show()
```



Key Observation

1. Frequent large withdrawal accounts: 19 (~9.9%)

A small but important group showing high outflow behaviour.

2. Overdraft accounts: 17 (~8.9%)

These accounts have negative balances, indicating financial instability.

3. Total risky accounts: 35 (~18.2%)

Nearly one-fifth of accounts exhibit risky financial behaviour.

4. Balanced contribution from both risk types

Large withdrawals (19) and overdrafts (17) contribute almost equally to risk.

5. Majority accounts (~81.8%) are not risky

Indicates overall dataset is relatively stable with limited high-risk exposure.

Key Insight

Risk is not widespread but concentrated in a small segment of accounts, making targeted monitoring effective.

4.2 Calculate balance volatility using standard deviation or coefficient of variation.

Standard Deviation: Measures how much balance varies from average

High Std Dev → high fluctuation

Coefficient of Variation (CV): $CV = \text{Standard Deviation} / \text{Mean}$

Normalized measure (better comparison across accounts)

Calculating Balance Volatility per Account

```
balance_volatility = df.groupby('AccountID').agg(  
    Avg_Balance = ('AccountBalance', 'mean'),  
    Std_Balance = ('AccountBalance', 'std')  
) .reset_index()
```

Calculating Coefficient of Variation (CV)

```
balance_volatility['CV'] = (  
    balance_volatility['Std_Balance'] / balance_volatility['Avg_Balance']  
)
```

Handling Edge Cases

```
balance_volatility['CV'] = balance_volatility['CV'].replace([np.inf, -np.inf], 0)  
balance_volatility['CV'] = balance_volatility['CV'].fillna(0)
```

Identify High Volatility Accounts

Using percentile threshold

```
vol_threshold = balance_volatility['CV'].quantile(0.75)  
  
high_volatility_accounts = balance_volatility[  
    balance_volatility['CV'] >= vol_threshold  
)
```

```
high_volatility_accounts.head()
```

	AccountID	Avg_Balance	Std_Balance	CV
2	ACC11062	57108.763888	40017.803232	0.700730
11	ACC15671	27077.566847	30995.763211	1.144703
21	ACC20297	80846.693440	70164.033043	0.867865
24	ACC21719	72818.112908	49213.497040	0.675841
26	ACC22036	74872.558018	57748.822230	0.771295

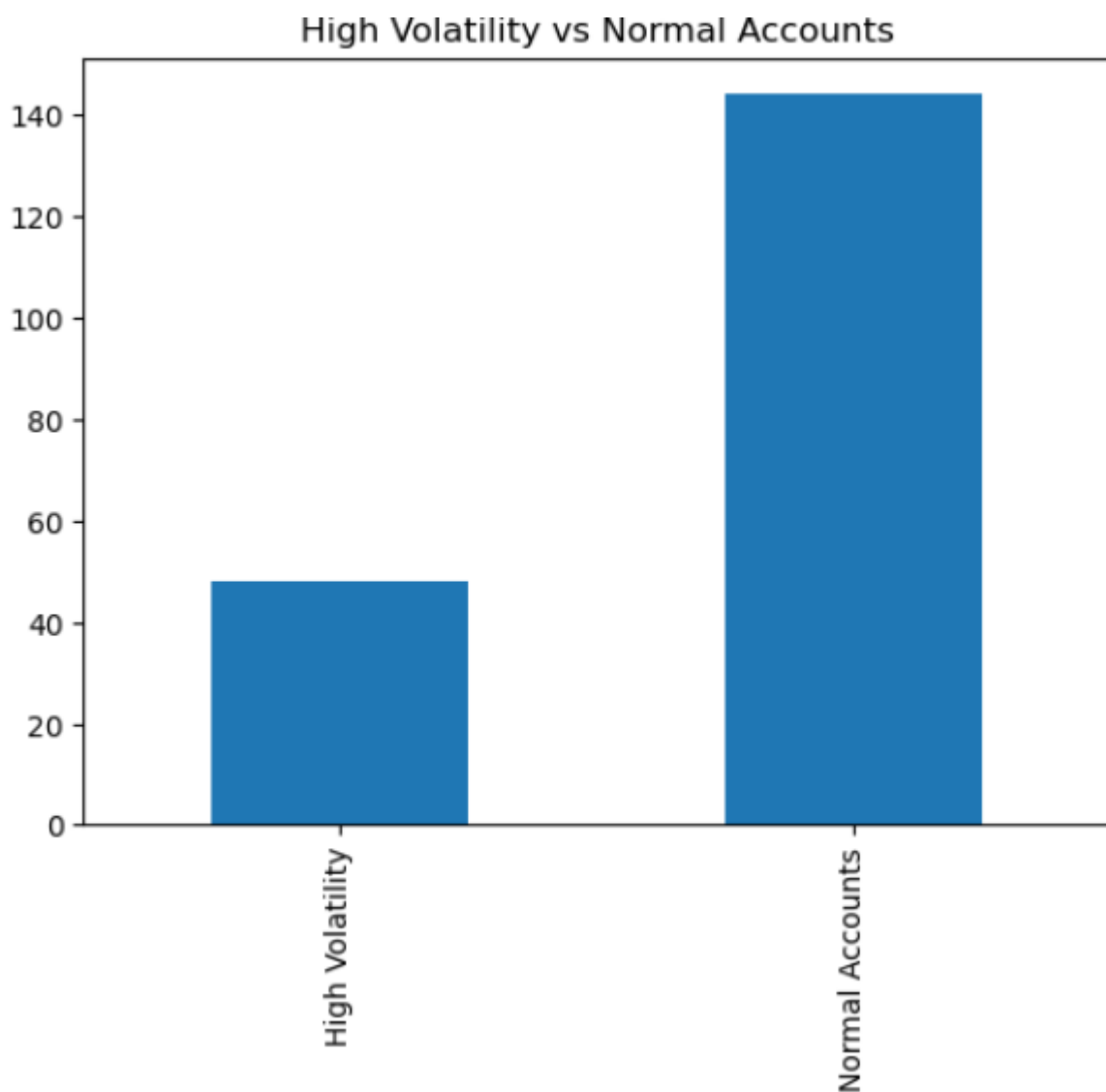
```
print("High Volatility Accounts:", len(high_volatility_accounts))
```

```
High Volatility Accounts: 48
```

Bar Chart

```
pd.Series({
    'High Volatility': 48,
    'Normal Accounts': 192-48
}).plot(kind='bar')

plt.title('High Volatility vs Normal Accounts')
plt.show()
```



Key Observations

1. High volatility accounts: 48 (~25%)

A quarter of accounts show high balance fluctuations.

2. Majority accounts (144, ~75%) are stable

Most customers maintain relatively consistent balances.

3. High CV values indicate unstable financial behaviour

Accounts with high CV show irregular inflow and outflow patterns.

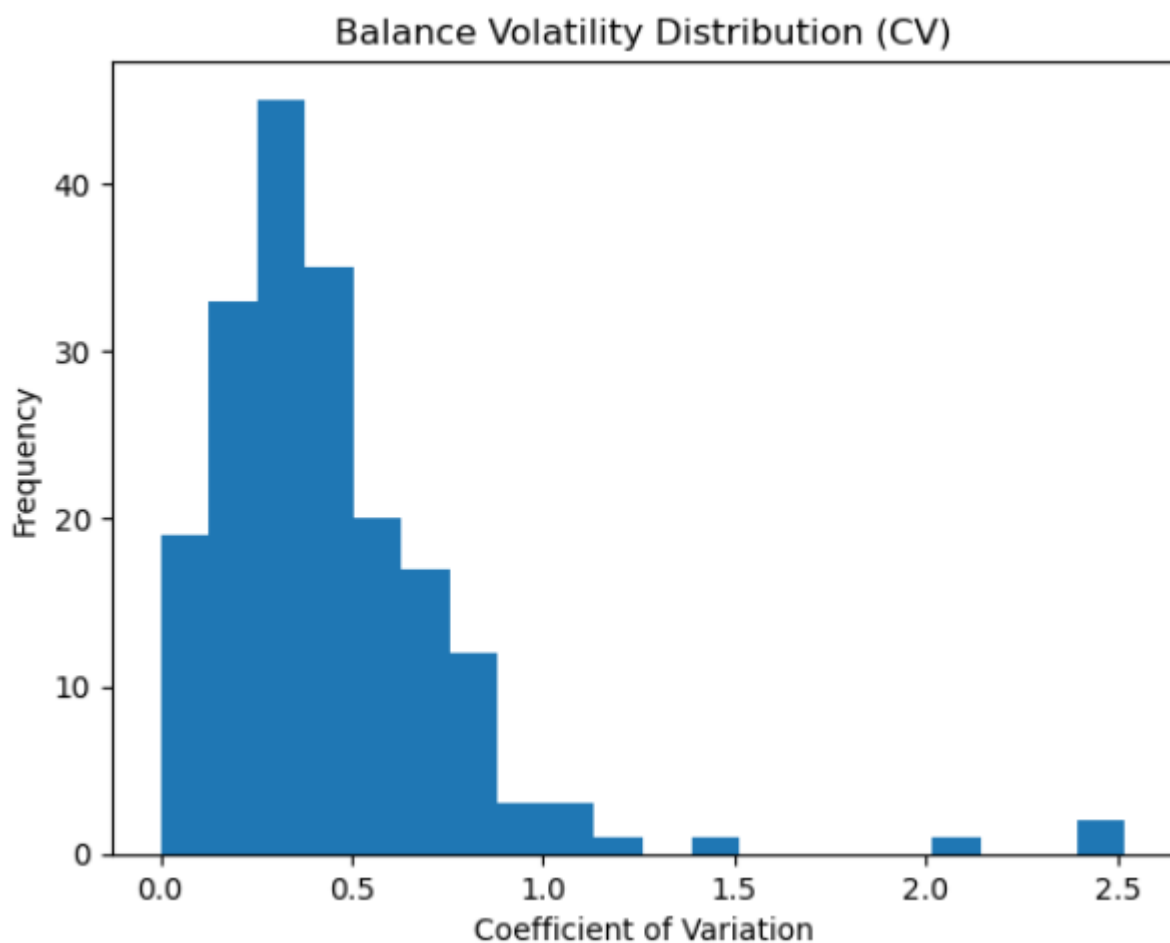
Key Insight

High balance volatility indicates inconsistent financial behaviour, which may signal financial instability or irregular cash flow patterns.

Now Using Histogram to show Distribution of volatility

```
balance_volatility['CV'].plot(kind='hist', bins=20)

plt.title('Balance Volatility Distribution (CV)')
plt.xlabel('Coefficient of Variation')
plt.show()
```



Key Observations (Balance Volatility Histogram)

1. Most accounts have low to moderate volatility (CV between ~0.2–0.6)

Indicates that the majority of customers maintain relatively stable balances.

2. Peak concentration around CV $\approx$ 0.3–0.5

This range represents the most common behaviour, showing consistent financial activity for most accounts.

3. Smaller number of high volatility accounts (CV > 0.75, 48 accounts ~25%)

These accounts show large fluctuations in balance and may indicate unstable financial patterns.

4. Right-skewed distribution observed

A long tail on the right side indicates fewer accounts with extremely high volatility (CV > 1.5).

5. Extreme outliers present (CV up to ~2.5)

These accounts have highly irregular balance behaviour and may require closer monitoring.

Key Insight

Most customers exhibit stable financial behaviour, while a small subset shows high volatility, indicating potential financial risk or irregular cash flow patterns.

4.3 Use IQR or z-score methods to detect anomalies.

Method 1: IQR (Interquartile Range)

$IQR = Q3 - Q1$

Lower Bound = $Q1 - 1.5 * IQR$

Upper Bound = $Q3 + 1.5 * IQR$

```
Q1 = df['TransactionAmount'].quantile(0.25)
Q3 = df['TransactionAmount'].quantile(0.75)

IQR = Q3 - Q1

lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

anomalies_iqr = df[
    (df['TransactionAmount'] < lower_bound) |
    (df['TransactionAmount'] > upper_bound)
]

len(anomalies_iqr)
```

7

Method 2: Z-Score

$Z = (X - \text{Mean}) / \text{Std Dev}$

Common Rule: $|Z| > 3 \rightarrow \text{Outlier}$

```
from scipy.stats import zscore

df['Z_score'] = zscore(df['TransactionAmount'])

anomalies_z = df[
    (df['Z_score'] > 3) | (df['Z_score'] < -3)
]

len(anomalies_z)
```

4

```
print("IQR Anomalies:", len(anomalies_iqr))
print("Z-score Anomalies:", len(anomalies_z))
```

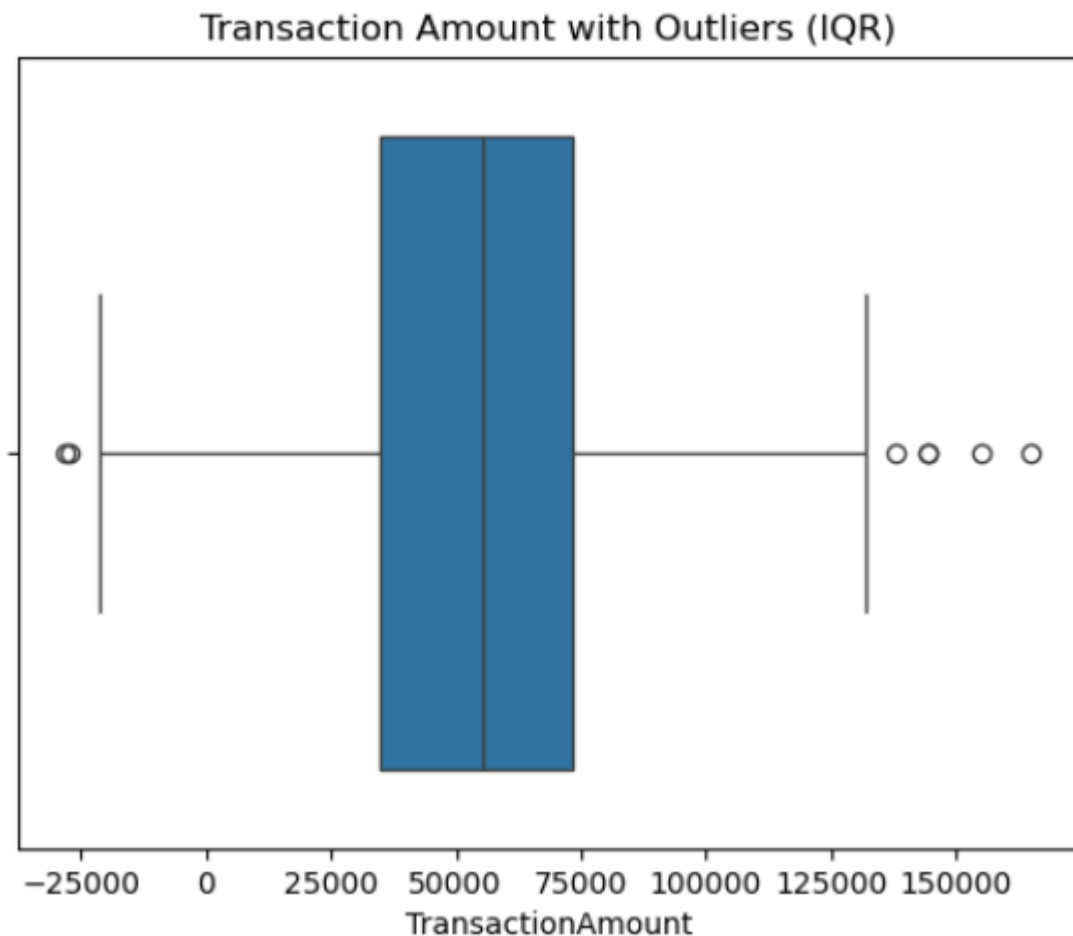
```
IQR Anomalies: 7
Z-score Anomalies: 4
```

Box Plot

```
import seaborn as sns
import matplotlib.pyplot as plt

sns.boxplot(x=df['TransactionAmount'])

plt.title('Transaction Amount with Outliers (IQR)')
plt.show()
```



Key observation (Anomaly Detection)

1. IQR method detected 7 anomalies (~3.6% of data)

These represent transactions significantly outside the normal range.

2. Z-score method detected 4 anomalies (~2.1% of data)

This method identifies extreme values based on standard deviation.

3. IQR captures more anomalies than Z-score

Indicates IQR is more sensitive to moderate outliers, while Z-score focuses on extreme ones.

4. Anomalies represent a very small portion of data (<5%)

Suggests overall transaction behaviour is relatively stable.

5. Detected anomalies indicate unusual transaction amounts

These could be high-value transactions, errors, or potentially fraudulent activity.

Key Strong Insight

Different methods capture different types of anomalies, highlighting the importance of using multiple approaches for robust outlier detection.

4.4 Highlight customers with irregular or suspicious transaction behaviour.

Single metric is not enough.

Real risk detection = combination of signals

We combine:

- Large withdrawals
- Overdrafts
- High volatility
- Anomalies

More signals = higher risk confidence

Risk Indicators Used

We will use:

- Frequent Large Withdrawals (Task 4.1)
- Overdraft Accounts (Task 4.1)
- High Volatility (Task 4.2)
- Anomalies (Task 4.3)

Converting All to Account Level

Large Withdrawal Accounts

```
large_withdrawal_accounts = set(frequent_large_withdrawals['AccountID'])
```

Overdraft Accounts

```
overdraft_accounts = set(df[df['AccountBalance'] < 0]['AccountID'])
```

High Volatility Accounts

```
volatility_accounts = set(high_volatility_accounts['AccountID'])
```

Anomaly Accounts

```
anomaly_accounts = set(anomalies_iqr['AccountID'])
```

Creating Risk Score

We assign a score based on how many risks factors an account has.

```
all_accounts = df['AccountID'].unique()

risk_df = pd.DataFrame({'AccountID': all_accounts})

risk_df['Large-Withdrawal_Flag'] = risk_df['AccountID'].isin(large_withdrawal_accounts).astype(int)
risk_df['Overdraft_Flag'] = risk_df['AccountID'].isin(overdraft_accounts).astype(int)
risk_df['Volatility_Flag'] = risk_df['AccountID'].isin(volatility_accounts).astype(int)
risk_df['Anomaly_Flag'] = risk_df['AccountID'].isin(anomaly_accounts).astype(int)
```

Calculating Total Risk Score

```
risk_df['Risk_Score_Total'] = (
    risk_df['Large-Withdrawal_Flag'] +
    risk_df['Overdraft_Flag'] +
    risk_df['Volatility_Flag'] +
    risk_df['Anomaly_Flag']
)
```

Identify Suspicious Customers

Condition: At least 2 risk factors

```
suspicious_customers = risk_df[risk_df['Risk_Score_Total'] >= 2]
suspicious_customers.head()
```

	AccountID	Large-Withdrawal_Flag	Overdraft_Flag	Volatility_Flag	Anomaly_Flag	Risk_Score_Total	Risk_Level
2	ACC11062	0	1	1	0	2	Medium Risk
11	ACC15671	0	1	1	0	2	Medium Risk
24	ACC21719	0	0	1	1	2	Medium Risk
26	ACC22036	0	1	1	0	2	Medium Risk
31	ACC24070	1	0	1	0	2	Medium Risk

```
print("Suspicious Customers:", len(suspicious_customers))
```

```
Suspicious Customers: 26
```

Risk Segmentation

```
def risk_level(x):
    if x >= 3:
        return 'High Risk'
    elif x == 2:
        return 'Medium Risk'
    else:
        return 'Low Risk'

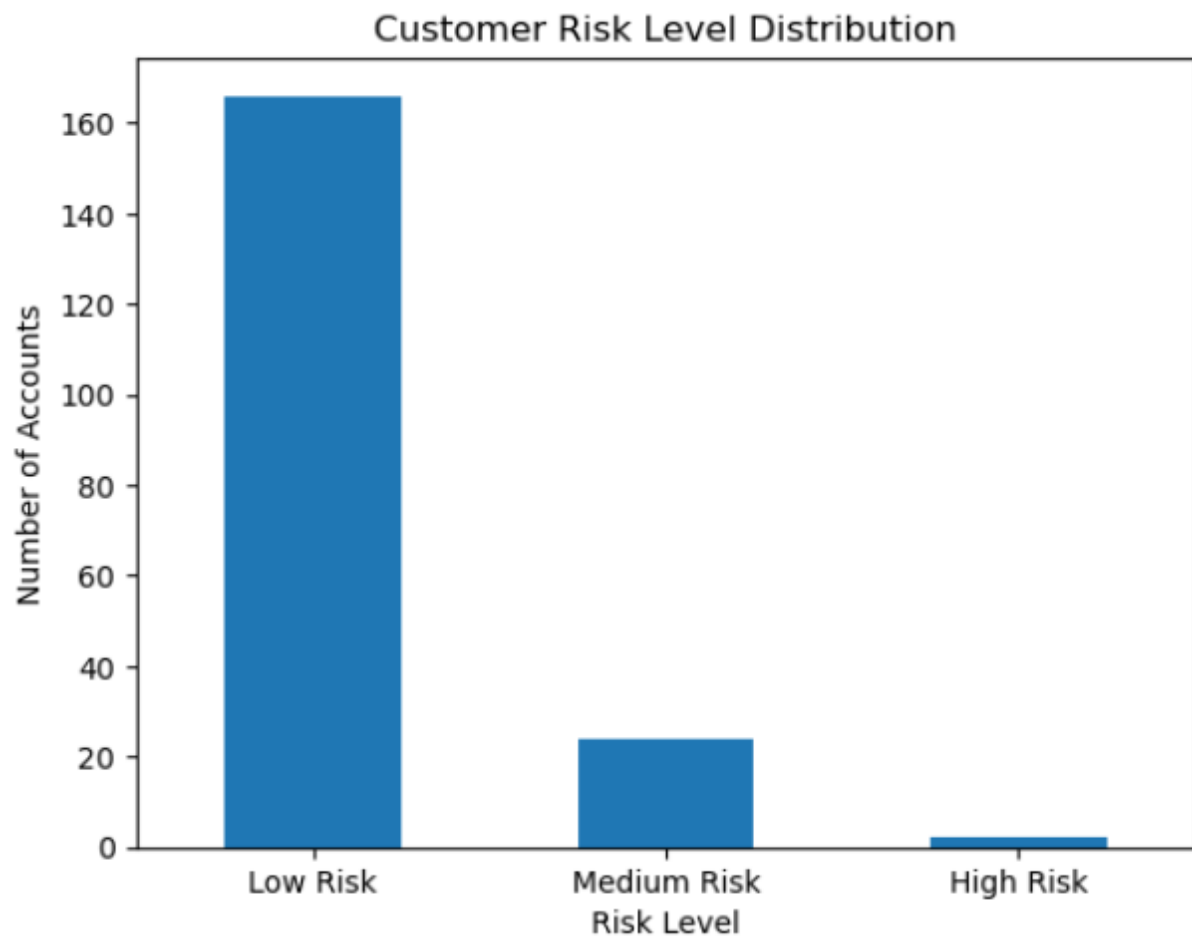
risk_df['Risk_Level'] = risk_df['Risk_Score_Total'].apply(risk_level)
```

```
risk_df['Risk_Level'].value_counts()
```

```
Risk_Level  
Low Risk      166  
Medium Risk   24  
High Risk      2  
Name: count, dtype: int64
```

Visualization

```
risk_df['Risk_Level'].value_counts().plot(kind='bar')  
  
plt.title('Customer Risk Level Distribution')  
plt.xlabel('Risk Level')  
plt.ylabel('Number of Accounts')  
plt.xticks(rotation=0)  
plt.show()
```



Key Observations

1. Majority accounts are Low Risk (166, ~86.5%)

Indicates overall customer base is financially stable.

2. Medium Risk accounts: 24 (~12.5%)

These accounts have multiple risk indicators and require monitoring.

3. High Risk accounts are very few (2, ~1%)

Represents a small but critical segment needing immediate attention.

4. Total suspicious customers: 26 (~13.5%)

Accounts with ≥ 2 risk factors indicate potential irregular or risky behaviour.

5. Risk is concentrated in a small segment

Most customers are stable, while risk is limited to a few high-impact accounts.

Key Insight

Combining multiple risk indicators provides a more accurate identification of suspicious customers compared to single-factor analysis.

Task 4: Financial Risk Identification (*Summary*)

Key Insights

1. Frequent large withdrawal accounts: 19 (~9.9%)

Indicates a segment with high cash outflow behavior.

2. Overdraft accounts: 17 (~8.9%)

These accounts show negative balances, reflecting financial instability.

3. High volatility accounts: 48 (~25%)

A quarter of accounts exhibit unstable balance patterns.

4. Anomalies detected: 7 (IQR) and 4 (Z-score) (<5%)

Outliers are limited but represent unusual or extreme transactions.

5. Suspicious customers: 26 (~13.5%), including 2 high-risk (~1%)

Risk is concentrated in a small segment with multiple risk factors.

Business Recommendations

1. Monitor high-risk accounts (26 accounts) closely

Implement alerts for accounts with multiple risk indicators.

2. Flag high volatility accounts (48 accounts)

Track irregular balance patterns to prevent potential financial issues.

3. Control large withdrawal behavior (19 accounts)

Introduce transaction limits or verification for high-value withdrawals.

4. Manage overdraft accounts (17 accounts)

Offer financial guidance or credit control mechanisms.

5. Use multi-factor risk scoring for decision-making

Combining multiple indicators improves accuracy in identifying risky customers.

Overall Summary

The financial risk analysis identified that while the majority of accounts are stable, approximately 13.5% exhibit risky behavior driven by large withdrawals, overdrafts, and volatility. Risk is concentrated in a small segment, enabling targeted monitoring and effective risk management strategies.

Task 5: Visualisation

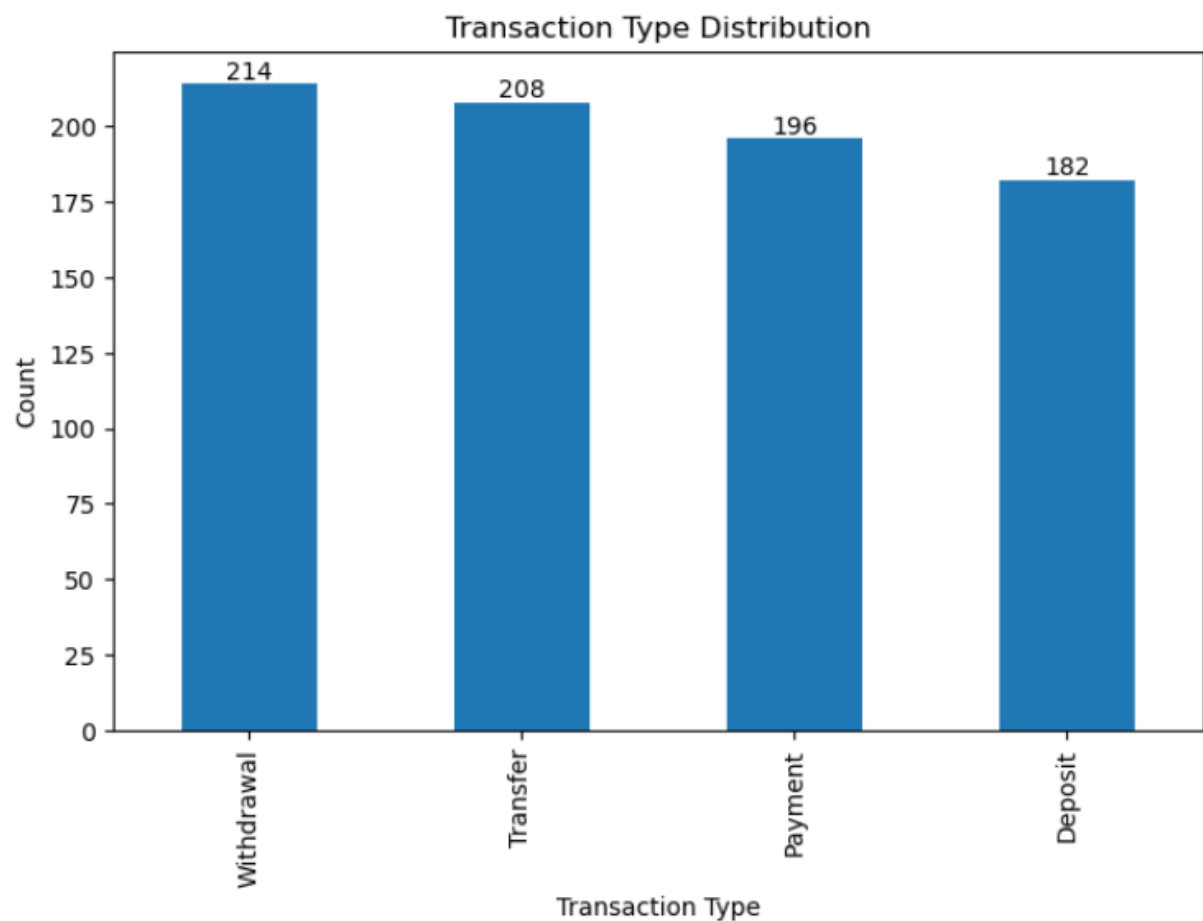
Conduct extensive exploratory data analysis with attractive visualizations for your findings

Note: EDA was performed throughout the project, starting from data cleaning and understanding, followed by trend analysis, segmentation, and risk analysis. Task 5 focuses specifically on visualization-based EDA.

Key visualizations such as Credit vs Debit trends, Customer Segmentation distribution, and Risk Level analysis were already developed in previous tasks. These charts are to support exploratory data analysis and highlight key patterns in customer behaviour and financial risk.

1. Transaction Type Distribution

```
ax = df['TransactionType'].value_counts().plot(kind='bar', figsize=(8,5))  
  
ax.bar_label(ax.containers[0])  
  
plt.title('Transaction Type Distribution')  
plt.xlabel('Transaction Type')  
plt.ylabel('Count')  
plt.show()
```



Key Insights (Transaction Type Distribution)

1. Withdrawals are highest (214 transactions, ~26.8%)

Indicates customers are more focused on cash outflow.

2. Transfers are second highest (208, ~26.0%)

Shows frequent fund movement between accounts.

3. Payments account for 196 (~24.5%)

Represents regular transactional usage like bills or purchases.

4. Deposits are lowest (182, ~22.8%)

Indicates comparatively lower inflow of funds.

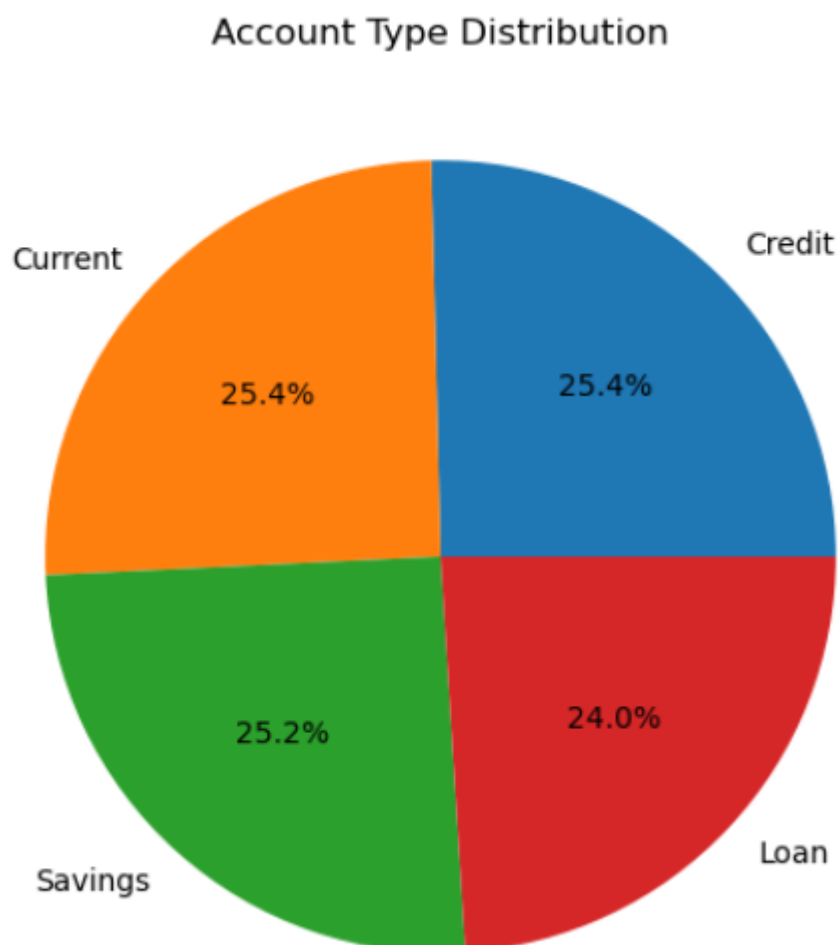
5. Overall distribution is fairly balanced

All transaction types are close in count, suggesting diversified customer activity.

2. Account Type Distribution

```
df['AccountType'].value_counts().plot(kind='pie', autopct='%1.1f%%', figsize=(6,6))

plt.title('Account Type Distribution')
plt.ylabel('')
plt.show()
```



Key Insights (Account Type Distribution)

1. Credit and Current accounts are highest (~25.4%)

Indicates slightly higher usage of these account types.

2. Savings accounts are close (~25.2%)

Shows strong preference for savings-based accounts.

3. Loan accounts are lowest (~24.0%)

Indicates slightly fewer customers using loan accounts.

4. All account types are nearly equal (~24–25%)

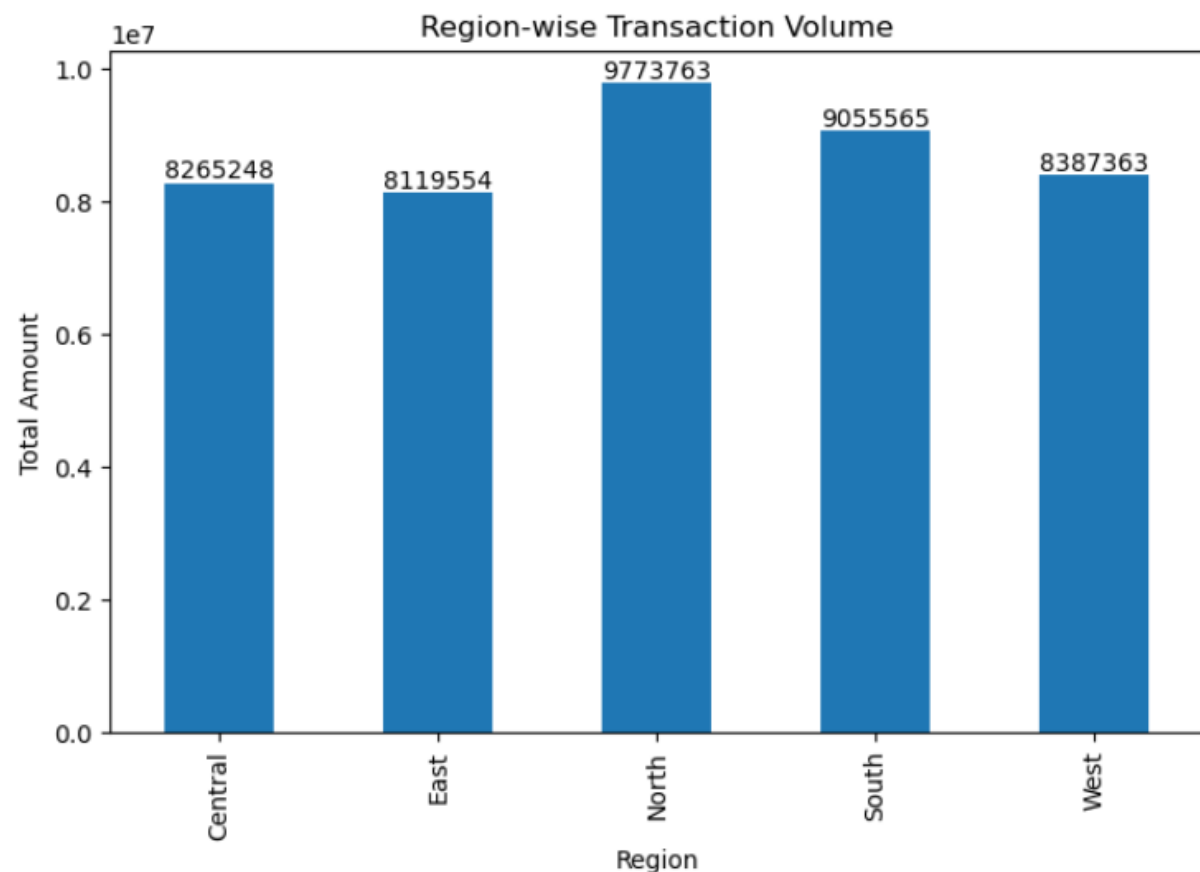
Suggests a well-balanced distribution across categories.

5. No dominant account type observed

Indicates a diversified and evenly spread customer base.

3. Region-wise Transactions

```
ax = df.groupby('Region')['TransactionAmount'].sum().plot(kind='bar', figsize=(8,5))  
  
ax.bar_label(ax.containers[0], fmt='%.0f')  
  
plt.title('Region-wise Transaction Volume')  
plt.ylabel('Total Amount')  
plt.show()
```



Key Insights (Region-wise Transaction Volume)

1. North region highest (~9.77M)

Indicates highest transaction activity in this region.

2. South region second (~9.06M)

Shows strong financial activity close to North.

3. West region moderate (~8.39M)

Represents stable but lower activity compared to top regions.

4. Central (~8.27M) and East (~8.12M) are lowest

Indicates comparatively lower transaction volumes.

5. Overall variation is moderate across regions

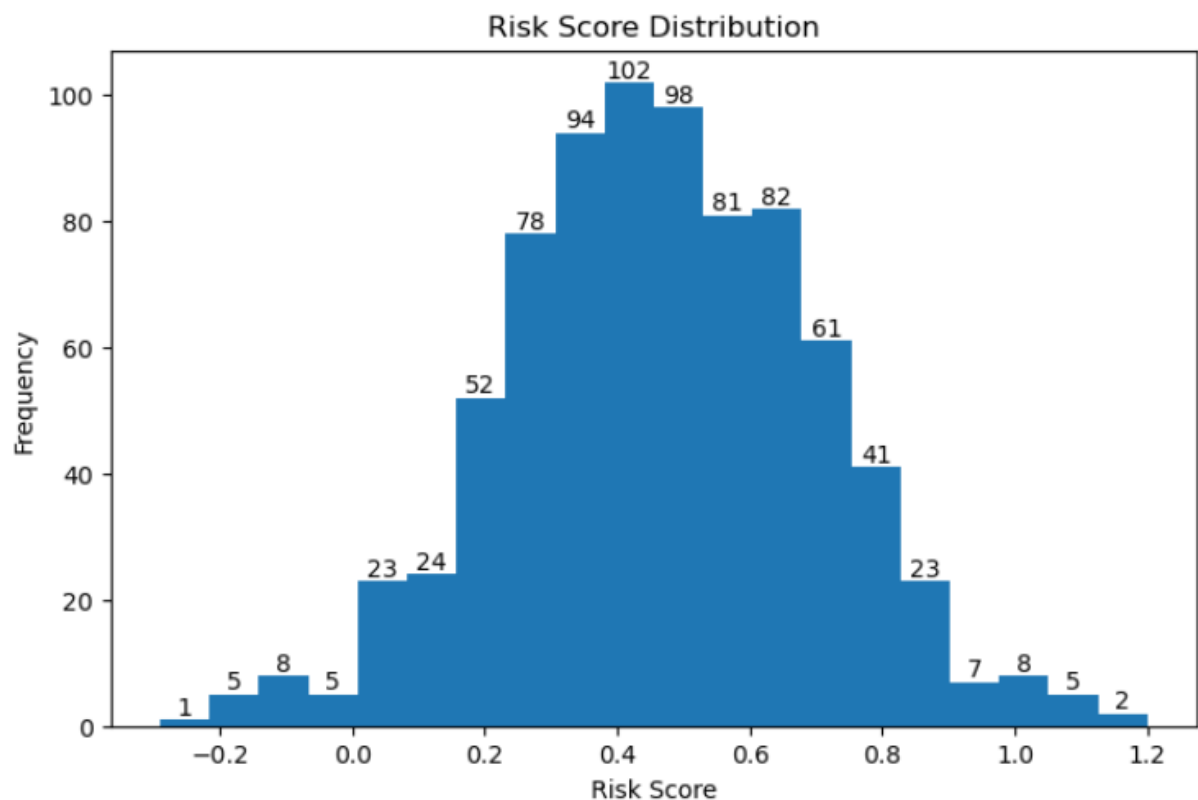
No extreme gaps, suggesting fairly balanced regional activity.

4. Risk Score Distribution

```
ax = df['RiskScore'].plot(kind='hist', bins=20, figsize=(8,5))

for container in ax.containers:
    ax.bar_label(container, fmt='%.0f')

plt.title('Risk Score Distribution')
plt.xlabel('Risk Score')
plt.show()
```



Key Insights (Risk Score Distribution)

1. Most scores concentrated between ~0.3–0.6

Indicates majority customers have moderate risk levels.

2. Peak around ~0.4–0.5 (highest frequency ~100+)

Shows most common risk range in dataset.

3. Very few extreme low (<0) and high (>1) scores

Indicates limited extreme risk cases.

4. Distribution is approximately normal (bell-shaped)

Suggests balanced spread of risk across customers.

5. High-risk tail exists but small (<10 counts)

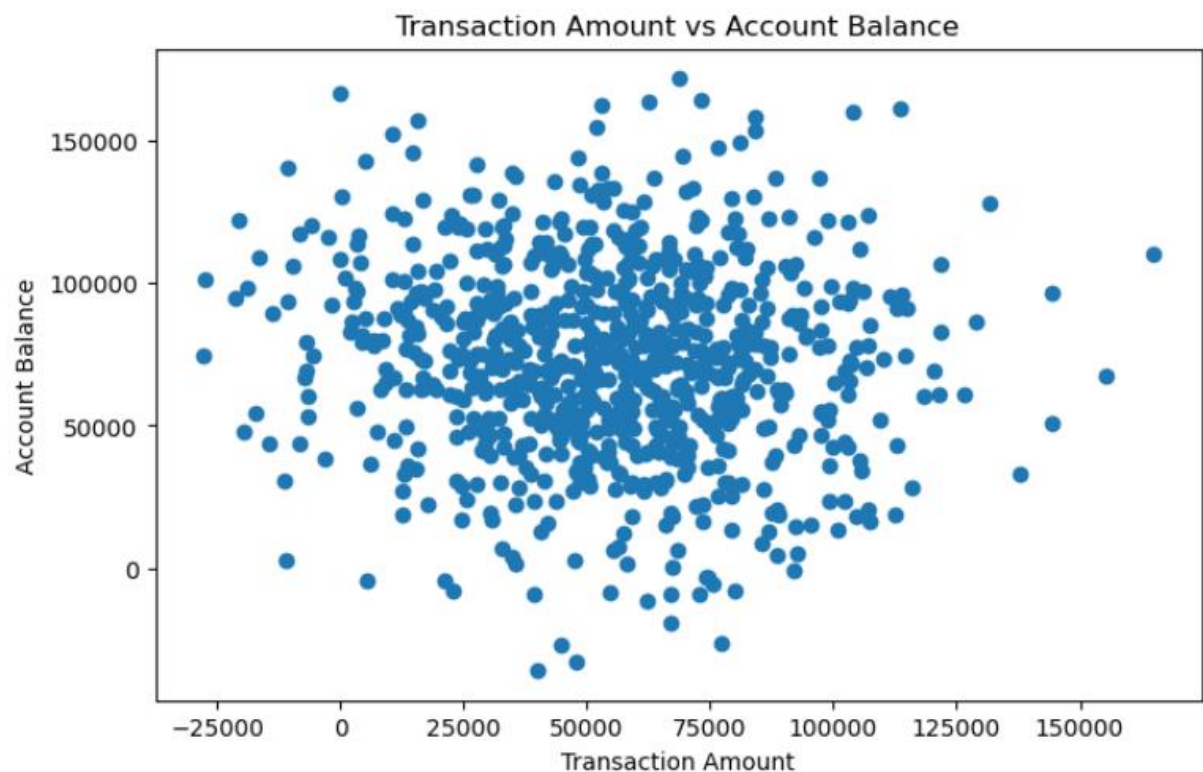
Only a few customers fall into very high-risk category.

5. Balance vs Transaction Amount

```
plt.figure(figsize=(8,5))

plt.scatter(df['TransactionAmount'], df['AccountBalance'])

plt.title('Transaction Amount vs Account Balance')
plt.xlabel('Transaction Amount')
plt.ylabel('Account Balance')
plt.show()
```



Key Insights (Transaction Amount vs Account Balance)

1. No strong linear relationship observed

Points are widely scattered, indicating weak correlation.

2. Most transactions cluster around 40K–80K amount

Represents the most common transaction range.

3. Balances mostly range between 40K–120K

Indicates typical account balance levels.

4. Some negative balance points observed

Indicates presence of overdraft accounts.

5. High transactions do not always mean high balance

Shows activity level is not directly linked to account strength.

6. Activity Level Distribution

```
ax = account_activity['Activity_Level'].value_counts().plot(kind='bar')  
  
ax.bar_label(ax.containers[0])  
  
plt.title('Customer Activity Levels')  
plt.show()
```

Key Insights (Customer Activity Levels)

1. Medium activity dominates (105, ~54.7%)

Indicates most customers have moderate transaction frequency.

2. High activity accounts (45, ~23.4%)

Represents smaller group of highly engaged users.

3. Low activity accounts (42, ~21.9%)

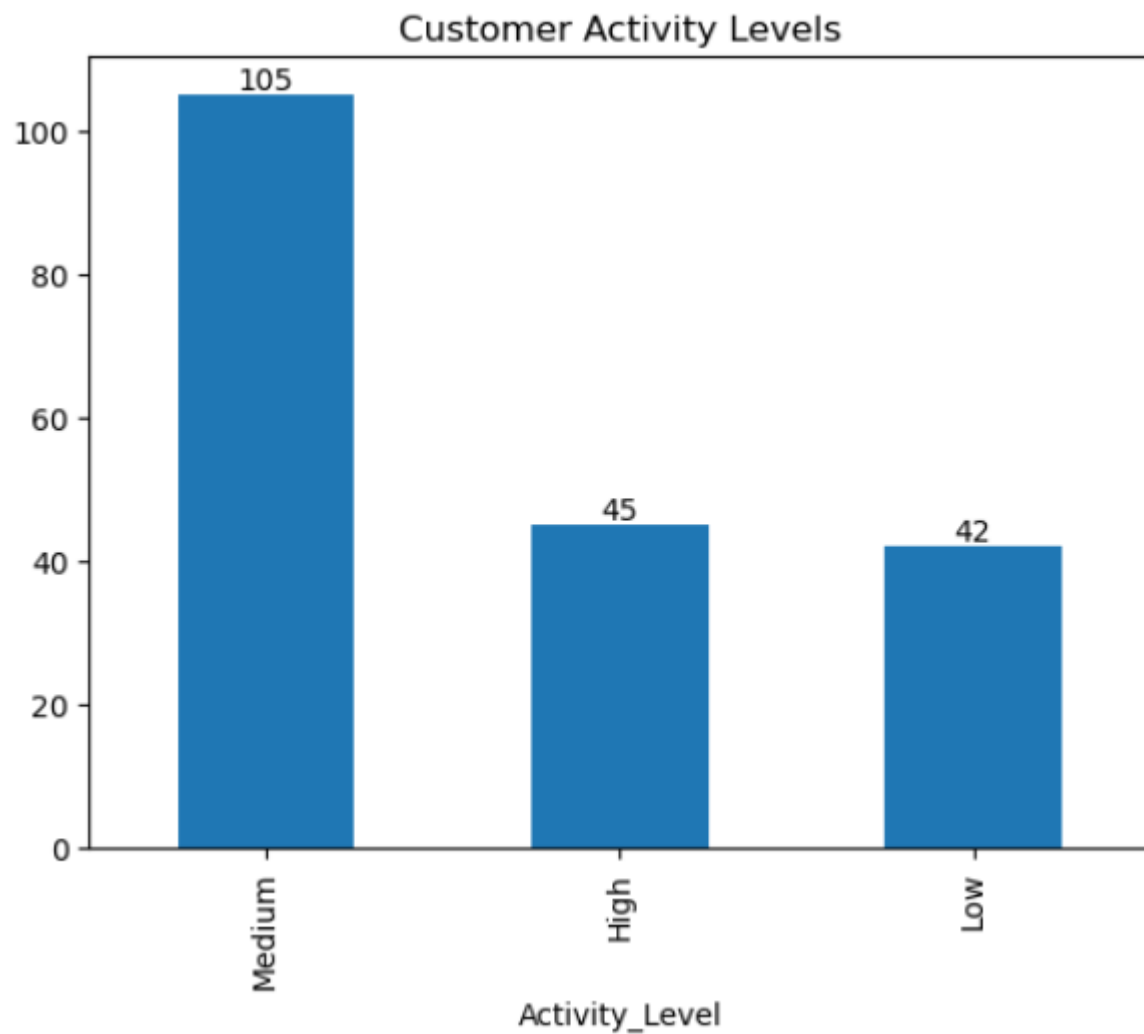
Indicates a portion of customers with low engagement.

4. Medium segment is more than double High/Low

Shows majority customers are neither very active nor inactive.

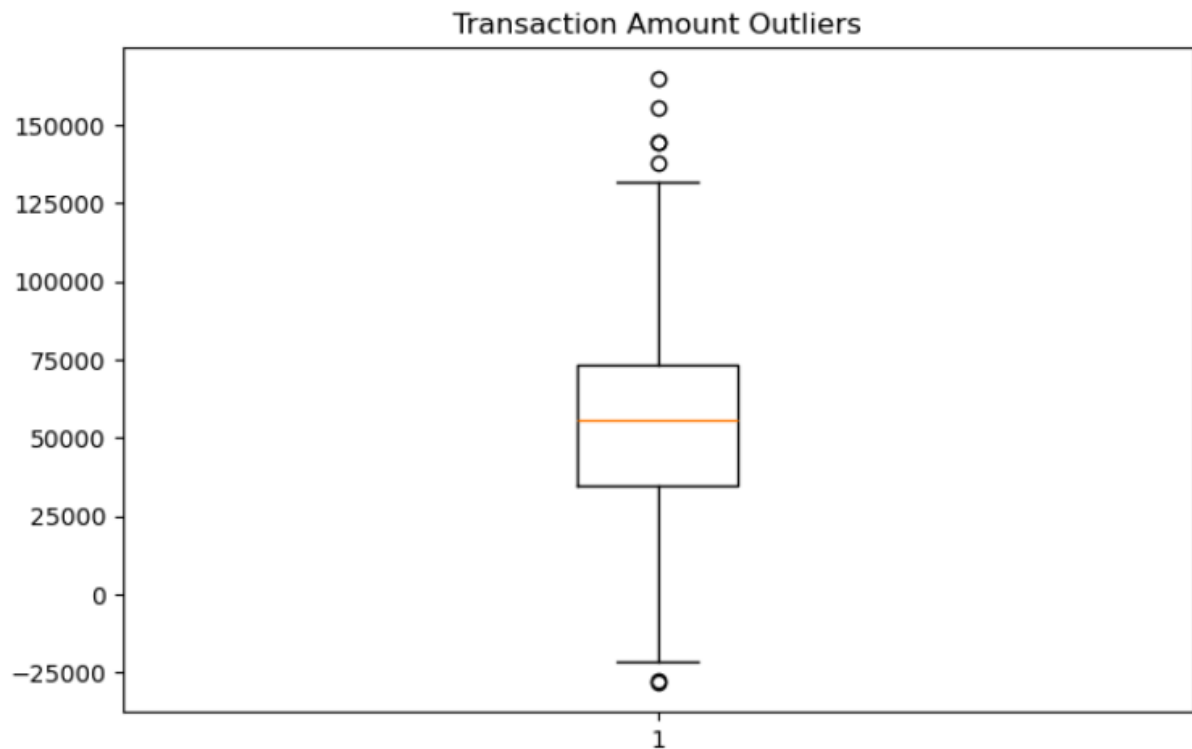
5. Balanced High vs Low segments (~45 vs 42)

Indicates similar proportion of highly active and less active users.



7. Boxplot for Outliers

```
plt.figure(figsize=(8,5))  
plt.boxplot(df['TransactionAmount'])  
plt.title('Transaction Amount Outliers')  
plt.show()
```

Key Insights (Transaction Amount Outliers)

1. Median transaction around ~55K

Indicates typical transaction amount level.

2. Most values lie between ~35K–75K (IQR range)

Represents normal transaction behavior

3. Upper outliers observed (>130K–170K)

Indicates presence of high-value unusual transactions.

4. Lower outliers also present (~-25K)

Suggests negative transactions or reversals/overdrafts.

5. Outliers are few compared to total data

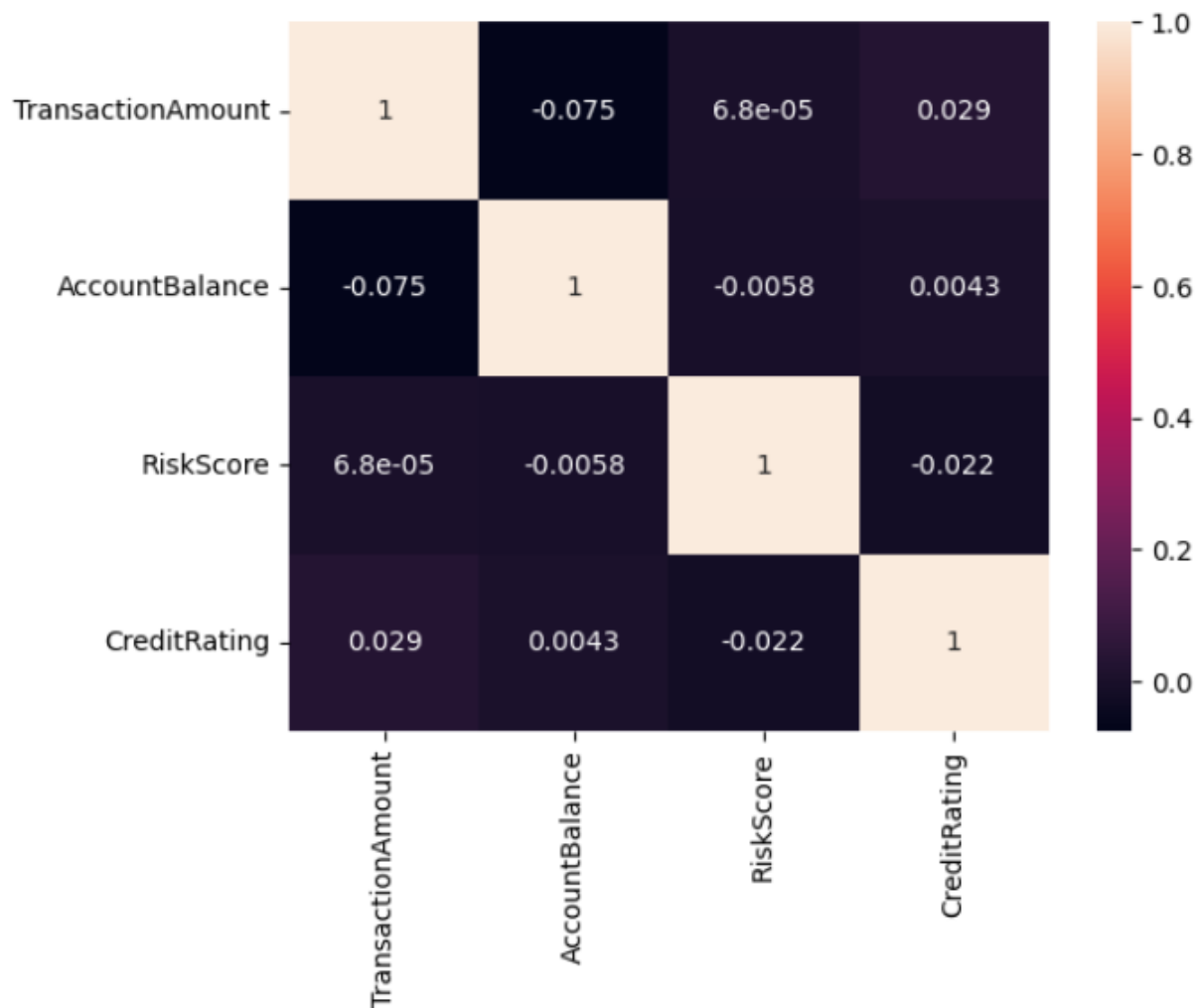
Indicates majority transactions follow normal range.

8. Heatmap

```
import seaborn as sns

corr = df[['TransactionAmount', 'AccountBalance', 'RiskScore', 'CreditRating']].corr()

sns.heatmap(corr, annot=True)
```

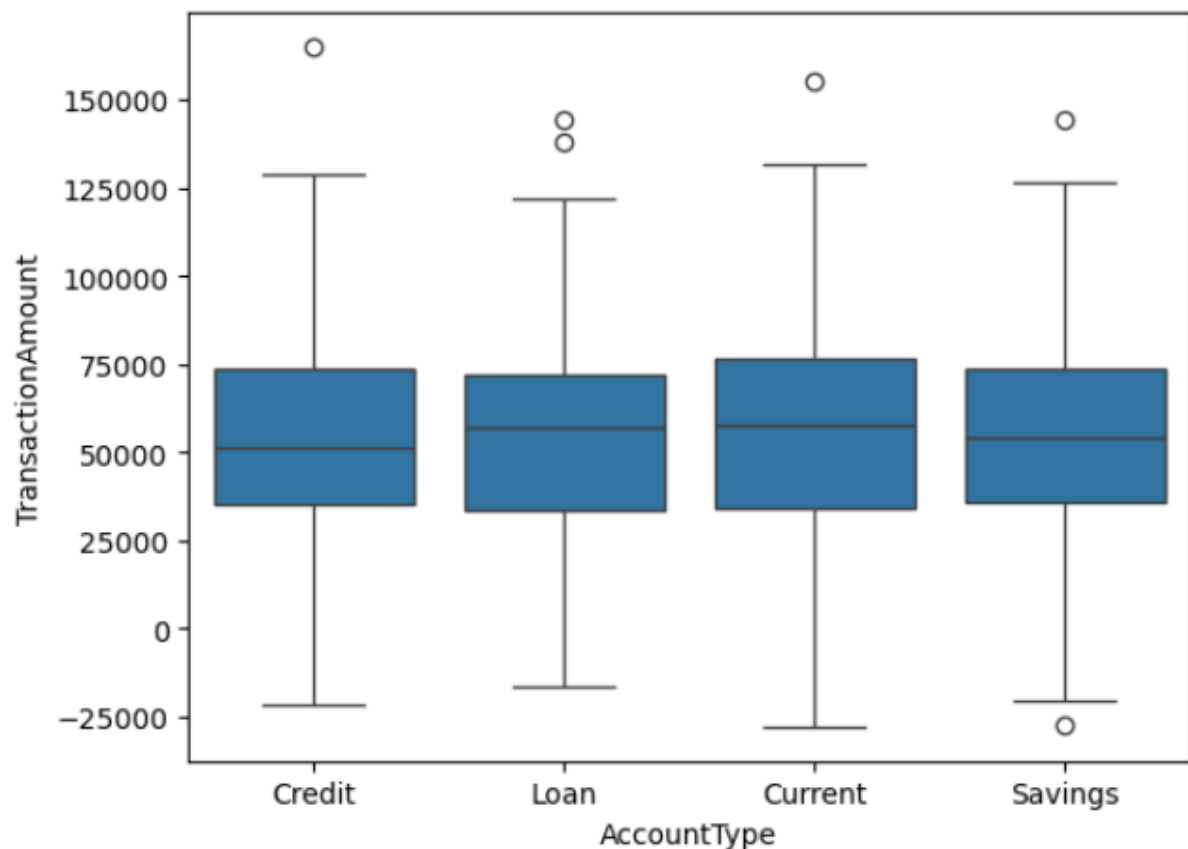


Key Insights (Correlation Heatmap)

- 1. Very weak correlation between all variables (values near 0)**
Indicates minimal linear relationships in dataset.
- 2. TransactionAmount vs AccountBalance slightly negative (-0.075)**
Suggests higher transactions do not increase balances.
- 3. RiskScore shows almost no correlation with other variables (~0)**
Indicates risk is independent of transaction amount and balance.
- 4. CreditRating has negligible correlation with all variables (<0.03)**
Suggests weak relationship with financial behavior metrics.
- 5. Overall, no strong predictive relationships observed**
Indicates variables behave independently in this dataset.

9. Boxplot by Account Type

```
sns.boxplot(x='AccountType', y='TransactionAmount', data=df)
```



Key Insights (Transaction Amount by Account Type)

1. Current accounts show slightly higher median (~60K)

Indicates relatively higher transaction activity.

2. Loan and Savings accounts have similar distributions (~55K median)

Shows consistent transaction behaviour across these types.

3. Credit accounts slightly lower median (~50K)

Indicates comparatively lower transaction amounts.

4. All account types show similar spread (IQR range)

Suggests uniform transaction variability across accounts.

5. Outliers present in all account types (>140K and <0)

Indicates occasional extreme transactions across categories.

Task 6: Hypothesis Testing

6.1 Test whether high-volume transaction accounts have statistically higher average balances than low-volume accounts.

Defining Hypotheses

Null Hypothesis (H_0) - There is NO difference in average balance between high-volume and low-volume accounts.

Alternative Hypothesis (H_1) - High-volume accounts have HIGHER average balance than low-volume accounts.

Using account profile dataset (from Task 3.2).

And Filtering High & Low Volume

```
high_volume = account_profile[
    account_profile['Transaction_Segment'] == 'High Volume'
]['Avg_Balance']

low_volume = account_profile[
    account_profile['Transaction_Segment'] == 'Low Volume'
]['Avg_Balance']
```

Performing Statistical Test

Using Independent T-Test

```
from scipy.stats import ttest_ind

t_stat, p_value = ttest_ind(high_volume, low_volume, equal_var=False)

print("T-statistic:", t_stat)
print("P-value:", p_value)
```

```
T-statistic: -1.909154168145703
P-value: 0.060066589508699246
```

Decision Rule

If $p\text{-value} < 0.05 \rightarrow \text{Reject } H_0$

If $p\text{-value} \geq 0.05 \rightarrow \text{Fail to reject } H_0$

Comparing Means

```
print("High Volume Mean:", high_volume.mean())
print("Low Volume Mean:", low_volume.mean())
```

```
High Volume Mean: 72816.5224060049
Low Volume Mean: 82143.28675136805
```

Decision Rule

If p-value < 0.05 → Reject H_0

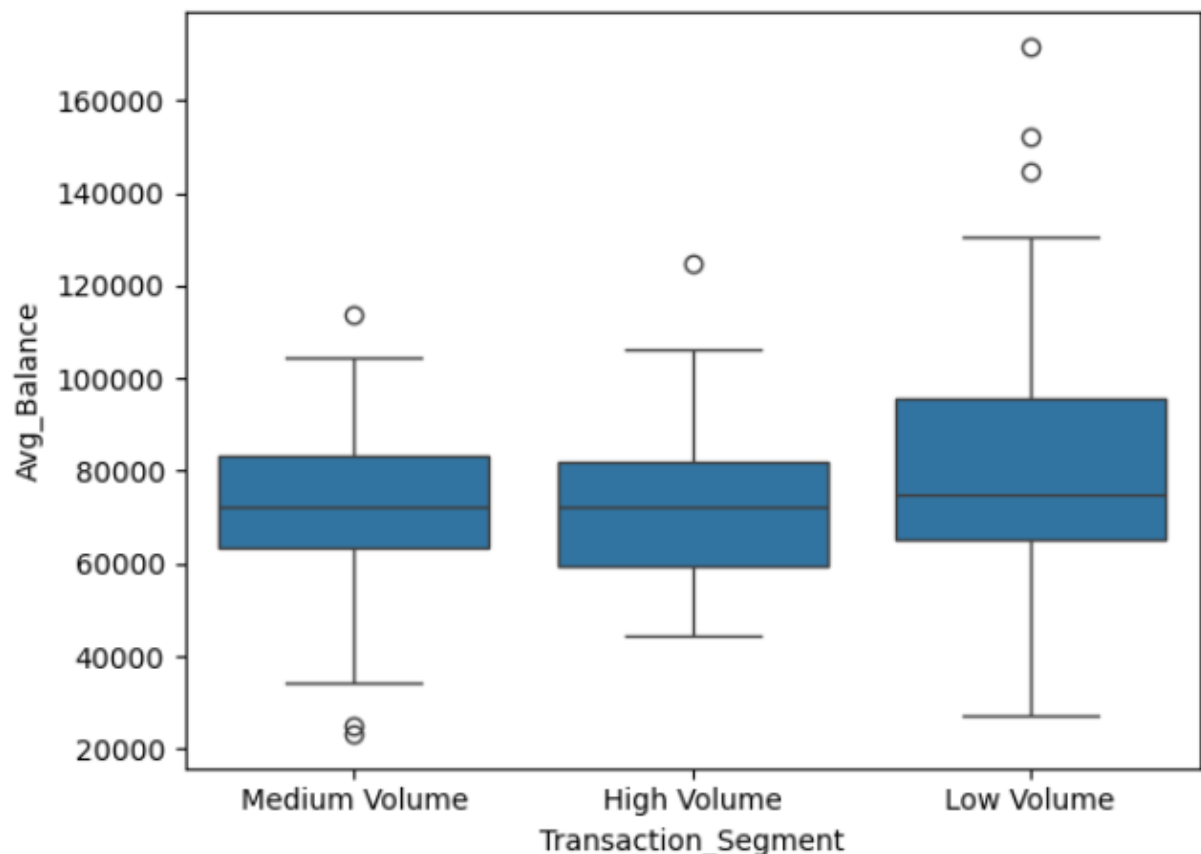
If p-value ≥ 0.05 → Fail to reject H_0

p-value = 0.060 (> 0.05)

The p-value is greater than 0.05, indicating no statistically significant difference between the two groups. Therefore, we fail to reject the null hypothesis.

Visualization

```
sns.boxplot(x='Transaction_Segment', y='Avg_Balance', data=account_profile)
```



Key Observations

1. **Low-volume accounts have higher average balance (~82K vs ~72K)**

Contradicts the initial assumption.

2. **p-value = 0.060 (>0.05)**

Indicates the difference is **not statistically significant**.

3. **Fail to reject null hypothesis**

No strong evidence that high-volume accounts have higher balances.

6.2 Conduct hypothesis testing based on segmentation.

Instead of 2 groups (High vs Low)

Now, we test across multiple segments

High Balance

Medium Balance

Low Balance

For this, we will use:

ANOVA Test (Analysis of Variance)

Defining Hypotheses

Null Hypothesis (H_0) - There is no significant difference in average transaction amounts across different balance segments.

Alternative Hypothesis (H_1) - At least one segment has a different average transaction amount.

Extracting Groups

```
high_bal = account_profile[
    account_profile['Balance_Segment'] == 'High Balance'
]['Total_Transaction']

medium_bal = account_profile[
    account_profile['Balance_Segment'] == 'Medium Balance'
]['Total_Transaction']

low_bal = account_profile[
    account_profile['Balance_Segment'] == 'Low Balance'
]['Total_Transaction']
```

Performing ANOVA Test

```
from scipy.stats import f_oneway

f_stat, p_value = f_oneway(high_bal, medium_bal, low_bal)

print("F-statistic:", f_stat)
print("P-value:", p_value)
```

```
F-statistic: 3.7969377932681567
P-value: 0.024170018244767177
```

Comparing Means

```
print("High Balance Mean:", high_bal.mean())  
print("Medium Balance Mean:", medium_bal.mean())  
print("Low Balance Mean:", low_bal.mean())
```

```
High Balance Mean: 185081.78531803898  
Medium Balance Mean: 241929.24061964583  
Low Balance Mean: 239424.1786288958
```

Decision Rule

If $p\text{-value} < 0.05 \rightarrow \text{Reject } H_0$

If $p\text{-value} \geq 0.05 \rightarrow \text{Fail to reject } H_0$

$p\text{-value} = 0.02 (< 0.05)$

The ANOVA test shows a statistically significant difference in transaction amounts across balance segments. Therefore, we reject the null hypothesis and conclude that customer segments behave differently.

Key Observation

1. $p\text{-value} = 0.024 (< 0.05)$

Indicates statistically significant difference between segments.

2. **Reject null hypothesis (H_0)**

Confirms that not all balance segments behave the same.

3. **At least one segment differs in transaction behaviour**

Shows segmentation has meaningful impact.

Final Summary: Customer Financial Behaviour & Risk Analysis

Key Business Answers

1. Customer Interaction with Account Types

- Account types are **evenly distributed (~24–25% each)** across Credit, Savings, Current, and Loan.
- Transaction behaviour is **consistent across account types**, with similar median transaction ranges (~50K–60K).
 - Indicates a **diversified and balanced customer base**.

2. Debit vs Credit Transaction Trends

- **Debits consistently exceed credits** across time periods.
- Withdrawals (214, ~27%) dominate over deposits (182, ~23%).
 - Indicates **higher cash outflow and potential financial pressure**.

3. Identification of High-Risk / Irregular Accounts

- **26 accounts (~13.5%) flagged as suspicious** (≥ 2 risk factors).
- Includes:
 - 19 large withdrawal accounts
 - 17 overdraft accounts
 - 48 high volatility accounts
 - Risk is **concentrated in a small segment**, enabling targeted monitoring.

4. Behaviour Linked to Low Balance / Overdraft

- **High-frequency low-balance accounts: 15 (~7.8%)**
- Weak correlation between balance & transaction amount (-0.075)
- Some accounts show **negative balances and high volatility**
Indicates **high activity does not guarantee financial stability**.

Overall Key Findings

- Majority accounts are **low risk (166, ~86.5%)**
- **~90% accounts dormant/inactive** → low engagement
- Transactions show **higher outflow vs inflow trend**
- Customer behaviour is **segmented but weakly correlated across variables**

Final Business Recommendations

1. **Target high-risk accounts (26)** with monitoring & alerts
2. **Reduce dormancy (~90%)** via engagement campaigns
3. **Control high withdrawals (19 accounts)** with limits/alerts
4. **Support overdraft users (17)** with financial guidance
5. **Leverage segmentation for personalized strategies**

Conclusion

The analysis shows that while most customers are stable, a small segment drives financial risk through high withdrawals, volatility, and overdrafts. Transaction patterns highlight consistent outflows and uneven engagement, enabling targeted interventions.

What Should They Do

Focus on **monitoring high-risk accounts and reducing dormancy through targeted engagement strategies.**

Use **segmentation and risk scoring to personalize services and improve financial stability across customers.**